VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BELAGAVI – 590 018, KARNATAKA

A PROJECT REPORT ON

INTELLIGENT ROAD ANOMALY DETECTION SYSTEM

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF ENGINEERING

in

ELECTRICAL AND ELECTRONICS ENGINEERING

(VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI)

Submitted by

Anurag R	**4SO23EE005**
Avaneesh H Kamath	**4SO23EE008**
Nikhil Nagesh Ambig	**4SO23EE031**
Rashmi M S	**4SO23EE038**

Under the Guidance of

Ms Himani Kishan Raj

Assistant Professor

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

(Accredited by National Board of Accreditation, New Delhi (Tier-I))

ST JOSEPH ENGINEERING COLLEGE

An Autonomous Institution

Accredited by NAAC with A+ Grade

Vamanjoor, Mangaluru - 575028, India

October 2026

St Joseph Engineering College

AN AUTONOMOUS INSTITUTION

Vamanjoor, Mangaluru – 575028

Department of Electrical and Electronics Engineering

(Accredited by National Board of Accreditation, New Delhi (Tier -I))

CERTIFICATE

Certified that the project work entitled **"Intelligent Road Anomaly Detection System"** is carried out by

Anurag R	**4SO23EE005**
Avaneesh H Kamath	**4SO23EE008**
Nikhil Nagesh Ambig	**4SO23EE031**
Rashmi M S	**4SO23EE038**

bonafide students of VII semester, B.E, Electrical and Electronics Engineering Department of St Joseph Engineering College, Vamanjoor, Mangaluru-575028, in partial fulfilment for the award of Bachelor of Engineering in Electrical and Electronics Engineering of the Visvesvaraya Technological University, Belagavi during the year 2026-27. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report and deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Ms Himani Kishan Raj	**Dr Sanath Saralaya**	**Dr Rio D'Souza**
Project Guide	Head of Department	Principal
Dept. of E&E Engineering	Dept. of E&E Engineering	

EXTERNAL VIVA VOCE EXAMINATION

1. Name:
   Designation: Signature with date

2. Name:
   Designation: Signature with date

Acknowledgement

We express our deep gratitude to Rev. Fr Wilfred Prakash D'Souza, Director, and Rev. Fr Kenneth Rayner Crasta, Assistant Director, for providing all the facilities for carrying out our project work.

We are indebted to our Principal, Dr Rio D'Souza, and the Management of St Joseph Engineering College, Vamanjoor, Mangaluru, for having provided all the facilities that helped us in the timely completion of the project.

We are grateful to Dr Sanath Saralaya, Head of the Department, Electrical and Electronics Engineering, and the Project Co-ordinator Ms Chaithra Shetty for her support and encouragement.

We would like to offer our earnest gratitude to our project guide, Ms Himani Kishan Raj, Assistant Professor, Department of Electrical and Electronics Engineering, SJEC, Mangaluru. This work would not have been possible without the encouragement and able guidance of her.

We wish to express our sincere gratitude to all the Faculty and Technical Staff of the Department of Electrical and Electronics Engineering, SJEC Mangaluru, for their valuable guidance, help and support.

Anurag R
Avaneesh H Kamath
Nikhil Nagesh Ambig
Rashmi M S

Abstract

Potholes and speed humps can cause sudden braking, vehicle damage, and accidents, while in-vehicle warnings do not alert nearby drivers. This project developed a low-cost system to detect road hazards in video, estimate their distance, recommend driver actions, and broadcast warnings to nearby vehicles.

About 1,000 labelled road images were augmented to 2,000. YOLOv8s and YOLOv8n were trained for 50 epochs, and YOLOv8s was selected for deployment. ByteTrack maintained persistent IDs so visible hazards could be tracked and counted once. MiDaS depth, lane position, and a position-based estimate were combined to classify hazards as `EMERGENCY_BRAKE`, `SLOW_DOWN`, or `MONITOR`. Hysteresis helped prevent alerts from changing in response to noisy distance estimates. An STM32 sent confirmed hazards in 6-byte packets over an nRF24L01 radio, and an ESP32 validated and displayed them on an OLED. The dashboard presented annotated video, estimated distance, hazard counts, and recommended actions.

On held-out images, YOLOv8s achieved mAP@0.5 of 0.40, precision of 0.90, and recall of 0.37. Since MiDaS provides relative rather than direct depth, distance estimates remained approximate. Hardware testing exposed radio failures caused by power noise and SPI timing; these were resolved with a regulated supply, decoupling capacitors, a slower SPI clock, and register read-back. The prototype demonstrates an integrated path from road-hazard detection to driver and wireless warnings, but it lacks standard V2V protocols, GPS, and message authentication, and cannot replace driver judgment.

Keywords: pothole detection, speed hump, YOLOv8, MiDaS, monocular depth, sensor fusion, ByteTrack, STM32, nRF24L01, ESP32, V2V communication.

List of Figures

4.1 Architecture of the proposed road-hazard detection and alert system 12
4.2 Flow chart of the complete system workflow 14
4.3 Data flow design of the proposed system 15
4.4 Between the STM32 Nucleo-L432KC and the nRF24L01 adapter 17
4.5 Wiring of the ESP32 with the nRF24L01 adapter and the SSD1306 OLED . . . 17
4.6 Structure of the 6-byte VANET alert packet and its radio frame 18

5.1 Dataset preparation and model training pipeline 19
5.2 Logic of the fusion engine for each tracked hazard 22
5.3 Layout of the virtual LiDAR window 24
5.4 Layout of the cockpit dashboard 24

7.1 Curves of the deployed YOLOv8s model 31
7.2 Confusion matrix of the deployed YOLOv8s model 32
7.3 Sample detections of potholes on road images 32
7.4 Sample detections of humps on road images 33
7.5 Complete hardware setup of the transmitter and receiver nodes 34
7.6 Receiver OLED displaying the hazard alerts, Example 1 34
7.7 Receiver OLED displaying the hazard alerts, Example 2 34
7.8 Cockpit dashboard and virtual LiDAR window during a detection 35

List of Tables

2.1 Hardware requirements 6
2.2 Software requirements 7

3.1 Comparison of existing systems 10

4.1 STM32 Nucleo-L432KC to nRF24L01 adapter connections 16
4.2 ESP32 connections to the nRF24L01 adapter and the SSD1306 OLED 16

5.1 Main fusion and pipeline parameters 23
5.2 STM32 peripheral configuration 25
5.3 nRF24L01 register configuration on the transmitter 26

6.1 Embedded and system test cases 28
6.2 Hardware integration issues and resolutions 29

7.1 Detection results on the held-out test set 31
7.2 End-to-end results of the alert chain 33
7.3 Comparison between this prototype and deployed V2V systems 36

List of Abbreviations

Abbreviation	Full Form
ADAS	Advanced Driver Assistance System
AP	Average Precision
BGR	Blue, Green, Red (colour channel order)
CE	Chip Enable
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CR LF	Carriage Return, Line Feed
CSN	Chip Select Not
DSRC	Dedicated Short-Range Communications
GPIO	General Purpose Input/Output
GPS	Global Positioning System
GPU	Graphics Processing Unit
HAL	Hardware Abstraction Layer
HUD	Head-Up Display
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoU	Intersection over Union
ISM	Industrial, Scientific and Medical (radio band)
ISR	Interrupt Service Routine
LiDAR	Light Detection and Ranging
MAE	Mean Absolute Error
mAP	Mean Average Precision
MiDaS	Mixed Datasets for Monocular Depth Estimation
MISO	Master In, Slave Out
MOSI	Master Out, Slave In
MSI	Multi-Speed Internal (oscillator)
OLED	Organic Light Emitting Diode
PLL	Phase-Locked Loop
RF	Radio Frequency
RX / TX	Receive / Transmit
SCK	Serial Clock
SDG	Sustainable Development Goal
SPI	Serial Peripheral Interface
UART	Universal Asynchronous Receiver-Transmitter
USART	Universal Synchronous/Asynchronous Receiver-Transmitter
V2V	Vehicle-to-Vehicle
V2X	Vehicle-to-Everything
VANET	Vehicular Ad Hoc Network
XOR	Exclusive OR
YOLO	You Only Look Once

Table of Contents

Acknowledgement i

Abstract ii

List of Figures iii

List of Tables iv

List of Abbreviations v

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Objectives . . . 2
- 1.4 Scope . . . 2

2 Software Requirements Specification 3
- 2.1 Introduction . . . 3
- 2.2 Problem Statement . . . 3
- 2.3 Scope . . . 3
- 2.4 Definitions and Abbreviations . . . 4
- 2.5 Functional Requirements . . . 5
- 2.6 Non-Functional Requirements . . . 5
- 2.7 Hardware Requirements . . . 6
- 2.8 Software Requirements . . . 6

3 Literature Survey 8
- 3.1 Road Anomaly Detection Using Vision . . . 8
- 3.2 Object Detection with YOLO . . . 8
- 3.3 Multi-Object Tracking . . . 8
- 3.4 Monocular Depth Estimation . . . 8
- 3.5 Sensor Fusion . . . 9
- 3.6 Vehicle-to-Vehicle Communication . . . 9
- 3.7 Embedded Wireless Platforms . . . 9
- 3.8 Summary of Literature Survey . . . 9
- 3.9 Comparison of Existing Systems . . . 10
- 3.10 Proposed System . . . 10
- 3.11 Research and Analysis . . . 10

4 System Design **12**
4.1 Architecture Design . . . 12
4.2 Decomposition Description . . . 13
4.3 Data Flow Design . . . 14
4.4 Hardware Design . . . 15
4.5 Packet and Protocol Design . . . 18

5 Implementation **19**
5.1 Dataset Preparation and Annotation . . . 19
5.2 Model Training on Google Colab . . . 19
5.3 Detection and Tracking Module . . . 20
5.4 Virtual LiDAR Depth Module . . . 20
5.5 Decision-Level Fusion Engine . . . 20
5.6 Dashboard and Visualisation . . . 23
5.7 Software-to-Hardware Bridge . . . 24
5.7.1 STM32 Firmware . . . 25
5.7.2 nRF24L01 Driver and Radio Configuration . . . 25
5.7.3 ESP32 Receiver and OLED Display . . . 26
5.8 Power and Signal-Integrity Measures . . . 26

6 System Testing **27**
6.1 Test Environment and Test Data . . . 27
6.2 Detection Evaluation Metrics . . . 27
6.3 Distance Validation Method . . . 27
6.4 Embedded and Wireless Link Testing . . . 28
6.5 Hardware Integration Issues and Resolutions . . . 28
6.6 Dashboard and Pipeline Testing . . . 30
6.7 Testing Summary . . . 30

7 Results and Discussion **31**
7.1 Detection Results . . . 31
7.2 Wireless Chain Results . . . 33
7.3 Discussion . . . 35

8 Conclusion and Future Work **37**
8.1 Conclusion . . . 37
8.2 Future Work . . . 37

References **39**

Sustainable Development Goals **41**

Chapter 1

Introduction

1.1 Background

The state of a road has a direct effect on how safe a trip is, how comfortable it feels, and how much it costs to keep a vehicle in good condition. Potholes and speed humps are among the most frequent surface problems drivers face. A pothole starts when water gets into the pavement and passing traffic gradually breaks the surface into a hollow. A speed hump is placed deliberately to slow vehicles, yet it is often unmarked or faded and easy to miss. If a driver notices either one late, the usual reaction is to brake sharply or swerve. This can lead to a loss of control, worn-out tyres and suspension, or a collision with the vehicle behind.

Advanced Driver Assistance Systems (ADAS) tackle this problem by using sensors fitted on the vehicle to observe the road and support the driver. Cameras are widely used for this purpose because they are low in cost and record a great deal of visual detail. When combined with deep learning, detectors such as YOLO [1], [2] can recognise and label road anomalies in real time. A camera has one clear limitation, however: a single image does not reveal how far away an object is. Monocular depth networks such as MiDaS [4] help overcome this by estimating relative depth from one image, and they can act as a "virtual LiDAR" when no physical range sensor is available. Once a system knows both what the hazard is and roughly how far it lies, the driver receives information that can be acted on.

An accurate onboard system still protects only the vehicle that carries it. Vehicle-to-Vehicle (V2V) communication [8] allows a car that has seen a hazard to alert others nearby, so they can prepare before the hazard comes into their own view. Commercial V2V systems depend on reserved radio bands and standardised message formats [9], [10]. This project follows a simpler path. A low-cost 2.4 GHz nRF24L01 radio link between an STM32 and an ESP32 microcontroller is used to show how a hazard warning can be broadcast on embedded hardware.

1.2 Problem Statement

Potholes and speed humps are difficult for a driver to predict, particularly at night, during rain or at high speed. Many inexpensive solutions only detect the anomaly. They neither report how far away it is nor suggest what the driver should do, and they do not share the warning with other road users. Measuring distance normally calls for LiDAR or radar, and both are costly.

This project therefore aimed to develop a system that could detect potholes and humps in real time from camera video and estimate their distance without any dedicated range hardware.

The system also needed to judge how serious each threat is and recommend a suitable driver action. Finally, it had to transmit the alert wirelessly to nearby receivers using affordable embedded hardware, and present the result clearly on both a dashboard and a receiver display.

1.3 Objectives

The main objectives of the project are:

1. To collect about 1,000 road images (potholes, humps and normal road), annotate them in YOLO format, and enlarge the dataset to about 2,000 images using augmentation.
2. To train YOLOv8s and YOLOv8n for 50 epochs each on Google Colab, compare the two, and select one for deployment.
3. To follow detected hazards through video frames with ByteTrack so that every hazard keeps a consistent ID.
4. To estimate the distance of each hazard using MiDaS monocular depth as a virtual LiDAR.
5. To build a decision-level fusion engine that merges detector output, depth and lane position into `EMERGENCY_BRAKE`, `SLOW_DOWN` or `MONITOR` decisions, using hysteresis to keep alerts steady.
6. To create a cockpit-style dashboard that shows the annotated video, distance, hazard counts, the recommended driver action, and a separate depth-map window.
7. To pass confirmed hazards from the laptop to an STM32 Nucleo-L432KC over UART, form a 6-byte packet, and broadcast it through an nRF24L01 radio.
8. To receive and check the packet on an ESP32 with a second nRF24L01, and show "POTHOLE AHEAD" or "HUMP AHEAD" on an SSD1306 OLED.
9. To assess the system through detection metrics, distance validation, hardware register verification and end-to-end testing.

1.4 Scope

This work is an academic prototype for detecting road hazards from a camera and broadcasting a wireless alert. It takes ordinary two-dimensional road video and detects two kinds of hazard: potholes and humps. Distance is found by combining relative monocular depth with a geometric estimate, and every hazard is then given a recommended action.
The embedded part consists of one transmitting node (STM32 with nRF24L01) and one receiving node (ESP32 with nRF24L01 and OLED). Any extra receiver set to the same radio settings can pick up the same broadcast.
The project does not include a certified automotive safety function, control of the vehicle, GPS positioning, standardised V2X messages or cryptographic authentication. The wireless link also uses the 2.4 GHz ISM band instead of the 5.9 GHz DSRC band, which limits its range. The system is intended for research, learning and demonstration, and it must not be used in place of the driver's own judgement.

Chapter 2

Software Requirements Specification

2.1 Introduction

Potholes and speed humps on the road are a frequent cause of vehicle damage and accidents. Computer vision and embedded systems can work together to spot these hazards early and pass a warning on to others. This chapter lays out the functional and non-functional requirements of the system, the terms used throughout the report, and the hardware and software needed to build and run it.

The proposed system has three main parts. The first is a perception pipeline on a laptop, which handles YOLOv8s detection, ByteTrack tracking, MiDaS depth estimation and decision-level fusion. The second is a UART link to an STM32 Nucleo-L432KC, which builds a packet and sends it out through an nRF24L01 radio. The third is an ESP32 receiver node fitted with an nRF24L01 and an SSD1306 OLED display.

2.2 Problem Statement

A camera fitted to a vehicle can see potholes and humps, but on its own it cannot say how far away they are or how quickly the driver must react. A detector can also give unsteady output from one frame to the next, which makes alerts flicker on and off. Even when a hazard is detected correctly, the warning stays inside a single vehicle unless it is sent onward.

The system must therefore handle three separate tasks: recognising the hazard, estimating its range and urgency, and delivering the alert. This project deals with them by training a custom detector and using a monocular depth network as a virtual range sensor. The two are combined at decision level with temporal latching, and the result is passed on through a low-cost embedded wireless chain.

2.3 Scope

The scope of this work is a working academic prototype. It accepts road video, detects potholes and humps, estimates their distance and classifies the threat level. The result is shown on a dashboard, and a 6-byte alert packet is sent over a 2.4 GHz broadcast link to an ESP32 node, which displays “POTHOLE AHEAD” or “HUMP AHEAD”.

The work covers dataset preparation, model training and comparison, tracking, depth estimation, fusion, dashboard development, STM32 and ESP32 firmware, radio configuration and hardware verification tests. It does not cover live on-road deployment, GPS positioning, authenticated V2X messaging or integration with vehicle controls. These capabilities can be added in future work.

2.4 Definitions and Abbreviations

Pothole: A hole in the road surface caused by water damage and traffic.

Speed hump: A raised bump on the road that makes vehicles slow down.

ADAS: Advanced Driver Assistance System; sensors and software that help the driver.

Object detection: Finding objects in an image and labelling them with boxes.

YOLO: You Only Look Once; a fast detector that scans the image in one pass.

YOLOv8s / YOLOv8n: Small and nano sizes of YOLOv8; small is more accurate, nano is faster.

Ultralytics: Python library used to train YOLOv8 and run ByteTrack.

ByteTrack: A tracker that keeps the same ID for an object across frames.

Monocular depth estimation: Estimating distance from a single camera image.

MiDaS: A pretrained depth model that gives relative, not metric, depth.

Virtual LiDAR: The MiDaS depth map used in place of a real LiDAR sensor.

Decision-level fusion: Merging the separate results of different sensing parts into one decision.

Hysteresis: Holding an alert briefly so it does not flicker.

V2V / V2X: Vehicle-to-Vehicle / Vehicle-to-Everything communication.

DSRC: Dedicated Short-Range Communications; a 5.9 GHz band used for V2V.

UART: Universal Asynchronous Receiver-Transmitter; a serial link (115200 baud, 8N1 here).

SPI: Serial Peripheral Interface; a four-wire bus for fast device communication.

I2C: Inter-Integrated Circuit; a two-wire bus used for the OLED.

STM32 Nucleo-L432KC: An ARM Cortex-M4 development board with a built-in debugger.

nRF24L01: A low-cost 2.4 GHz radio module controlled over SPI.

Auto-Acknowledge: A feature where the receiver replies to each packet; switched off here.

ESP32: A dual-core microcontroller with Wi-Fi and Bluetooth, used as the receiver.

SSD1306: Controller chip of the 128×64 OLED display.

Packet / checksum: A fixed block of bytes sent together / an extra byte to detect corruption.

Precision, Recall, mAP: Scores for correct detections, objects found, and average accuracy over classes.

IoU: Intersection over Union; overlap between predicted and true boxes.

Epoch: One full pass of the training data through the model.

Data augmentation: Making altered copies of images to improve training.

2.5 Functional Requirements

1. **Video input:** The system shall take road video from a file or a camera.
2. **Hazard detection:** The trained YOLOv8s model shall detect potholes and speed humps, using a confidence threshold of 0.70 and an IoU threshold of 0.45.
3. **Object tracking:** ByteTrack shall give every detected hazard a persistent ID.
4. **Depth estimation:** MiDaS shall produce a dense depth map for the processed frames.
5. **Distance and lane estimation:** The system shall estimate each hazard's distance in metres and place it in the left, centre or right lane.
6. **Threat classification:** Each hazard shall be labelled `EMERGENCY_BRAKE`, `SLOW_DOWN` or `MONITOR`, with hysteresis so the alert does not flicker.
7. **Dashboard:** The system shall show the annotated video, fused distance, recommended driver action and a rolling detection log, along with a separate depth-map window.
8. **Hazard counting:** Each unique hazard track shall be counted once per session, as either a pothole or a hump.
9. **UART command:** The laptop shall send the text command `POTHOLE` or `HUMP`, ending in CR LF, to the STM32 at 115200 baud, at most once every 1.5 seconds.
10. **Packet construction:** The STM32 shall build a 6-byte packet holding a system token, vehicle ID, alert code, frame counter, XOR checksum and a reserved byte.
11. **Wireless broadcast:** The STM32 shall send the packet through the nRF24L01 on channel 80 with address NODE1 and Auto-Acknowledge off, transmitting each alert three times.
12. **Reception and validation:** The ESP32 shall check the token (0xAA) and the checksum, and discard any invalid packet.
13. **Driver alert:** The ESP32 shall show POTHOLE AHEAD or HUMP AHEAD on the OLED for three seconds, then return to the waiting screen.
14. **Diagnostics:** The STM32 shall print the nRF24L01 register values over UART at start-up so the radio setup can be checked.

2.6 Non-Functional Requirements

1. **Real-time performance:** The pipeline shall run on a laptop CPU by running the heavy models on every second frame and reusing the tracks in between.
2. **Accuracy:** The detector shall be judged on held-out images using precision, recall and mAP, and the distance estimate shall be checked against measured distances.
3. **Reliability:** Every radio register write shall be verified by reading it back, and each alert shall be sent more than once because no acknowledgements are returned.

4. **Power robustness:** Each nRF24L01 adapter shall run from a stable supply with 100 μF and 10 μF capacitors fitted close to it.
5. **Interoperability:** The transmitter and receiver shall share the same radio settings and the same packed packet structure.
6. **Multiple receivers:** Any number of receivers on the same channel and address shall be able to pick up the same broadcast.
7. **Modularity:** Detection, tracking, depth, fusion, dashboard and packet code shall sit in separate files so each can be changed on its own.
8. **Configurability:** Thresholds, the resend interval and the camera parameters shall be kept together in one configuration file.
9. **Usability:** The dashboard shall give the recommended action as one clear phrase.
10. **Portability:** The software shall run on Windows with Python, the STM32 firmware shall build in STM32CubeIDE, and the ESP32 sketch shall build in the Arduino IDE.
11. **Security limitation:** The packet only detects accidental corruption, so the system shall not be described as authenticated or encrypted.
12. **Responsible use:** The system shall be presented as an academic prototype that does not replace the driver's judgement or a certified safety system.

2.7 Hardware Requirements

Table 2.1: Hardware requirements

Component	Specification	Purpose
Laptop / PC	Multi-core CPU, USB port, 12500H i5 12th gen, 16 GB RAM	Runs the perception pipeline and dashboard
STM32 Nucleo-L432KC	ARM Cortex-M4, 32 MHz sys-tem clock, ST-LINK virtual COM port	UART bridge, packet builder, radio driver
nRF24L01 with adapter (2 units)	2.4 GHz, 1 Mbps, SPI; adapter with onboard regulator (5 V input)	Wireless transmitter and receiver
ESP32 DevKit	Dual-core Xtensa, SPI and I2C	Receiver controller
SSD1306 OLED	128×64, I2C address 0x3C	Driver alert display
Capacitors	100 μF and 10 μF electrolytic	Supply decoupling at each radio adapter
Jumper wires, USB cables	Direct wiring, no breadboard for SPI lines	Interconnection and power

2.8 Software Requirements

The perception pipeline is written in Python and uses OpenCV [15] for video input, drawing and display, and PyTorch [16] to run the MiDaS model. Table 2.2 lists the software used.

Table 2.2: Software requirements

Software	**Version / Library**	**Purpose**
Operating system	Windows	Host for the perception pipeline
Python	Python 3 with NumPy	Perception and dashboard code
OpenCV	`cv2`	Video input, drawing, display windows
PyTorch	`torch` (CPU)	MiDaS inference
Ultralytics	YOLOv8 and ByteTrack	Training, detection, tracking
MiDaS_small	`intel-isl/MiDaS` via `torch.hub`	Monocular depth estimation
pyserial	`serial`	UART link to the STM32
STM32CubeIDE	STM32 HAL for the L4 series	STM32 firmware
Arduino IDE	RF24, Adafruit_GFX, Adafruit_SSD1306	ESP32 firmware
Google Colab	GPU runtime	Model training
PuTTY	Serial terminal	STM32 debug output and manual tests

Chapter 3

Literature Survey

3.1 Road Anomaly Detection Using Vision

Researchers have tried several ways of sensing road-surface problems. Some systems use the accelerometer and gyroscope in a smartphone to feel a bump as the vehicle crosses it [6]. They are inexpensive, but they react only after the vehicle has already hit the anomaly. Vision-based methods look at the road ahead, so they can warn the driver before contact. Dhiman and Klette [5] studied pothole detection using computer vision and learning. The results of camera-based methods depend heavily on how good and how varied the training images are.

3.2 Object Detection with YOLO

Redmon et al. [1] introduced YOLO, a network that predicts bounding boxes and class probabilities in one pass over the image, which made real-time detection practical. The Ultralytics YOLOv8 family [2] comes in several sizes, from nano to extra-large, and all of them share the same training and inference interface. The smaller models, such as YOLOv8n, run faster, while the larger ones, such as YOLOv8s, are usually more accurate. This project trains both YOLOv8s and YOLOv8n on a custom road dataset and deploys the small version.

3.3 Multi-Object Tracking

Detections can appear and disappear from frame to frame, so a tracker is used to give each object a stable identity. Zhang et al. [3] proposed ByteTrack, which also uses low-confidence boxes when matching detections to existing tracks, instead of discarding them. ByteTrack can be used directly through the Ultralytics tracking interface. This project uses it so that every pothole or hump keeps one ID while it is visible and is counted only once.

3.4 Monocular Depth Estimation

Ranftl et al. [4] proposed MiDaS, a depth network trained on a mixture of datasets so that it works well across different scenes. It predicts relative inverse depth. It can tell which pixels are nearer or farther, but it does not give a distance in metres, and its scale can change from one image to another. This project uses MiDaS_small as a virtual LiDAR. Its values are converted to approximate metres through a fixed mapping and blended with a geometric estimate. Driving benchmarks such as KITTI [7] are often used to evaluate depth estimation on road scenes, which is the setting of this project.

3.5 Sensor Fusion

Sensor fusion means combining information from more than one source. In decision-level (late) fusion, each branch first reaches its own conclusion and the conclusions are merged afterwards. This keeps the branches independent and easy to swap, but it throws away detail that earlier fusion could have used. Here, the appearance branch (YOLO) and the range branch (MiDaS) are combined after each has produced its output, and a temporal latch extends the fusion across frames.

3.6 Vehicle-to-Vehicle Communication

V2V communication lets vehicles share safety information with each other directly. Kenney [8] describes the DSRC standards used in the United States, where vehicles send messages in the 5.9 GHz band. The SAE J2735 standard [9] defines the message set, including the Basic Safety Message, and IEEE 1609.2 [10] covers the security services that authenticate messages. Real deployments therefore rely on reserved spectrum, standard messages with position data, and cryptographic protection. This project borrows only the broadcast warning idea of V2V and builds it on commodity hardware.

3.7 Embedded Wireless Platforms

The nRF24L01+ [11] is an inexpensive 2.4 GHz transceiver controlled over SPI. Its Enhanced ShockBurst protocol provides hardware address matching, CRC and optional auto-acknowledgement. The STM32 Nucleo-32 boards [12] offer an ARM Cortex-M4 microcontroller with a built-in ST-LINK that appears as a virtual COM port, and the ESP32 [13] is a low-cost microcontroller widely used in embedded work. Small monochrome OLED displays based on the SSD1306 controller [14] are driven over I2C. Together these parts make an affordable platform for a V2V-style alert chain.

3.8 Summary of Literature Survey

The reviewed work supports the main building blocks of this project: single-pass detection (YOLO), robust tracking (ByteTrack), monocular depth (MiDaS) and the broadcast idea behind V2V. Earlier work usually treats these separately. This project joins them into one chain, from a camera frame to a driver-facing alert on a second wireless node, and it states openly where the prototype differs from deployed V2V standards.

3.9 Comparison of Existing Systems

Table 3.1: Comparison of existing systems

Ref.	Study	Method	Relevance to this project	Limitation
[1], [2]	YOLO, Ultralytics YOLOv8	Single-pass CNN detectors	Basis of the hazard detector	2D boxes only; accuracy depends on the training data
[3]	ByteTrack	Tracking by associating every detection box	Persistent hazard IDs	Gives no distance information
[4]	MiDaS	Mixed-dataset monocular depth	Virtual LiDAR depth map	Relative depth, scale-ambiguous
[5]	Computer-vision pothole detection	Vision and learning-based detection	Supports camera-based pothole detection	Detection-centred; alert sharing is outside its focus
[6]	Smartphone road monitoring	Inertial sensors	Alternative to vision	Reactive: senses the anomaly on contact
[8]–[10]	DSRC, SAE J2735, IEEE 1609.2	5.9 GHz V2V, standard messages, security	Defines real V2V	Needs certified hardware, not used in the prototype
[11]	nRF24L01+	2.4 GHz SPI transceiver	Low-cost wireless link	Short range, shared ISM band, no security

3.10 Proposed System

The proposed system is an academic prototype that connects camera-based hazard detection to a wireless alert. A YOLOv8s detector trained on a custom pothole and hump dataset finds the hazards, ByteTrack gives each one a persistent ID, and MiDaS supplies a depth map. A decision-level fusion engine then combines these with lane position to estimate distance and label the threat as `EMERGENCY_BRAKE`, `SLOW_DOWN` or `MONITOR`. A hysteresis latch keeps the high-threat state on for nearby hazards so that alerts stay stable.

When a hazard needs action, the laptop sends a `POTHOLE` or `HUMP` command over UART to an STM32 Nucleo-L432KC. The STM32 builds a 6-byte packet and broadcasts it three times per alert through an nRF24L01 on channel 80, with Auto-Acknowledge disabled. An ESP32 receiver checks the token and checksum and shows POTHOLE AHEAD or HUMP AHEAD on an OLED. The dashboard displays the annotated video, the distance, the session pothole and hump counts, and the recommended driver action: CLEAR ROAD, BRAKE, SLOW DOWN, or TURN LEFT / TURN RIGHT when the hazard is in a neighbouring lane.

3.11 Research and Analysis

The research part looks at four things. First, YOLOv8s and YOLOv8n are trained for the same 50 epochs on the same dataset and compared using precision, recall and mAP, and the better trade-off is deployed. Second, the fused distance is checked against measured distances to see how far the blended estimate can be trusted. Third, the wireless chain is tested for register correctness, packet integrity and reception, and its delay from command to display is measured.

Fourth, the hardware problems met during integration, such as power-supply noise and SPI signal integrity, are recorded along with their fixes.

On the held-out test images, the deployed YOLOv8s detector achieved a mAP@0.5 of 0.40, precision of 0.90 and recall of 0.37, compared with 0.40 mAP@0.5 for YOLOv8n. The application part packs the pipeline into one program with two windows, and the embedded part packs the radio chain into two small firmware images. The project does not claim that the depth-based distance is metrically calibrated, that the broadcast link follows V2V standards, or that the system is fit to serve as a safety function.

Chapter 4

System Design

4.1 Architecture Design

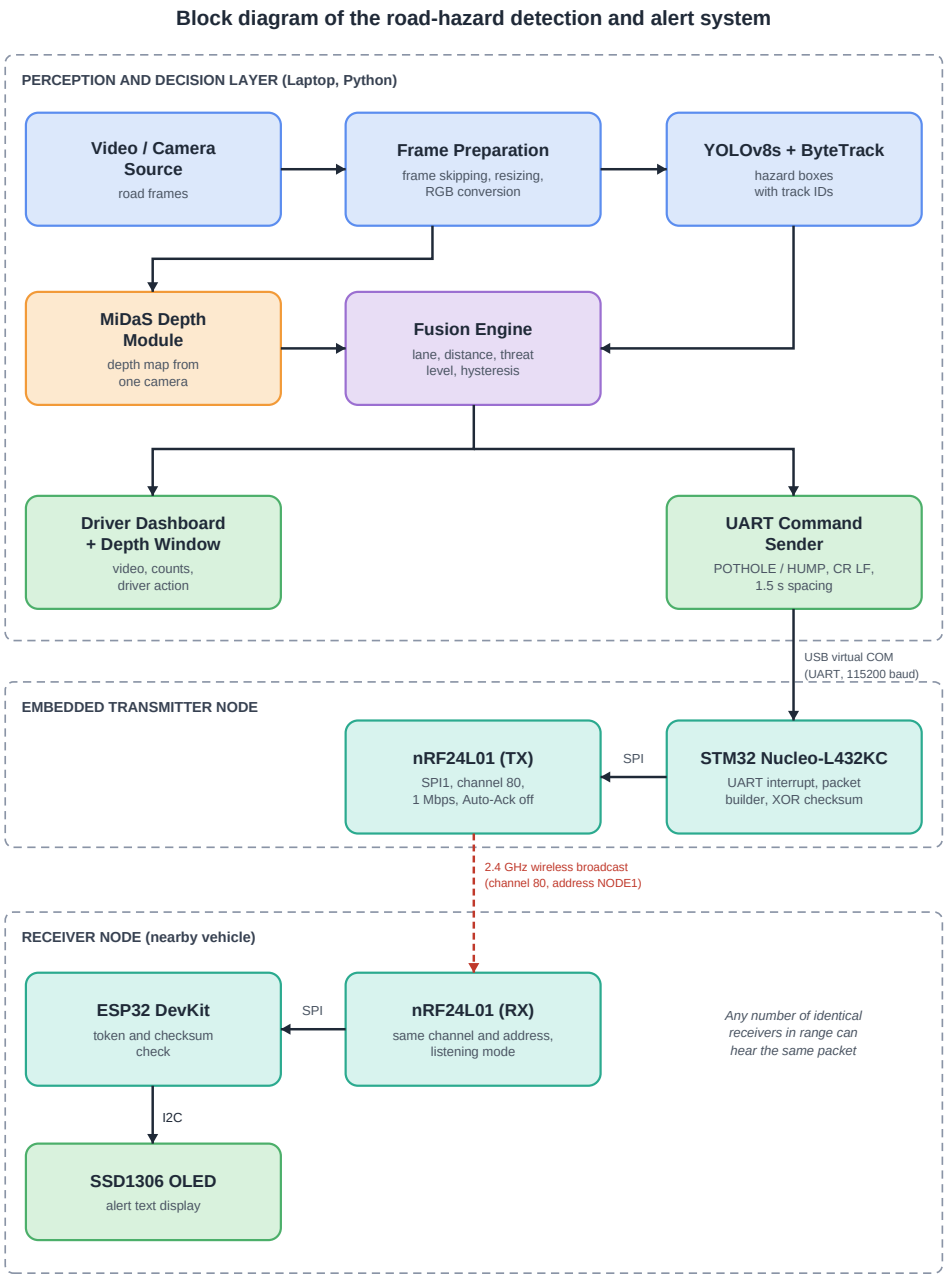

Figure 4.1: Block diagram of the road-hazard detection and alert system

Figure 4.1: Architecture of the proposed road-hazard detection and alert system

The system is split into three layers. The first is the perception and decision layer, which runs on a laptop. It reads the video, resizes each frame, and passes it to YOLOv8s with ByteTrack to find and follow potholes and humps. MiDaS builds a depth map in parallel, and a decision-level fusion engine joins both results to estimate distance and decide the threat level. The output feeds the cockpit dashboard and the UART command sender.
The second layer is the embedded transmitter node, built around an STM32 Nucleo-L432KC. It receives the text command through the USB virtual COM port, forms a 6-byte packet, and sends it out through an nRF24L01 radio.
The third layer is the receiver node, an ESP32 with its own nRF24L01. It checks the incoming packet and shows the alert on an SSD1306 OLED. The wireless hop is a broadcast on channel 80, so any receiver with the same settings can pick it up.

4.2 Decomposition Description

The system is decomposed into the following modules:

1. Dataset preparation, annotation and augmentation module
2. Model training and evaluation module (Google Colab, Ultralytics)
3. Detection module (`detector.py`) and tracking module (`tracker.py`)
4. Virtual LiDAR depth module (`depth_estimator.py`)
5. Decision-level fusion module (`fusion.py`)
6. Dashboard module (`hud.py`) and main control loop (`main.py`)
7. Telemetry packet module (`packet_generator.py`) and UART command sender
8. STM32 firmware: UART receiver, packet builder, nRF24L01 driver
9. ESP32 firmware: nRF24L01 receiver, validator, OLED display

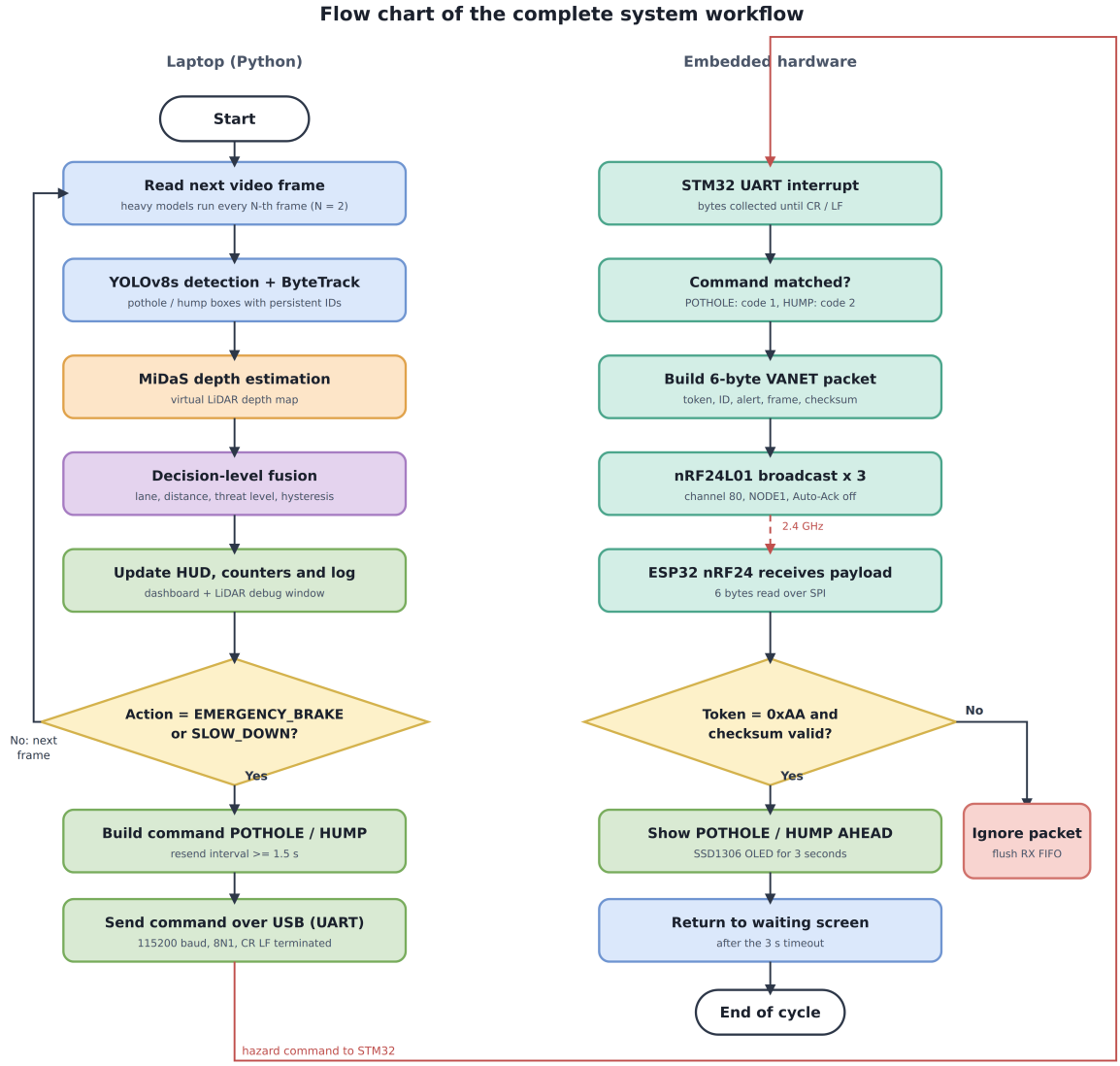

Figure 4.2: Flow chart of the complete system workflow

The workflow in Figure 4.2 shows how the laptop and the embedded hardware cooperate. The laptop reads a frame, runs the heavy models on every second frame, fuses the results, updates the dashboard, and sends a command only when the action is `EMERGENCY_BRAKE` or `SLOW_DOWN`. The STM32 turns the command into a packet and broadcasts it; the ESP32 either displays the alert or ignores an invalid packet.

4.3 Data Flow Design

The data flow begins when a road video frame is read. Frame data goes to detection and tracking (process 2.0) and to depth estimation (process 3.0). Both outputs go to the fusion process (4.0), which produces tracks with distance, lane and threat level. These feed the dashboard (5.0) and the UART command generator (6.0). The STM32 (7.0) converts the command into a radio packet, and the ESP32 (8.0) receives it and shows the alert to the receiving driver.

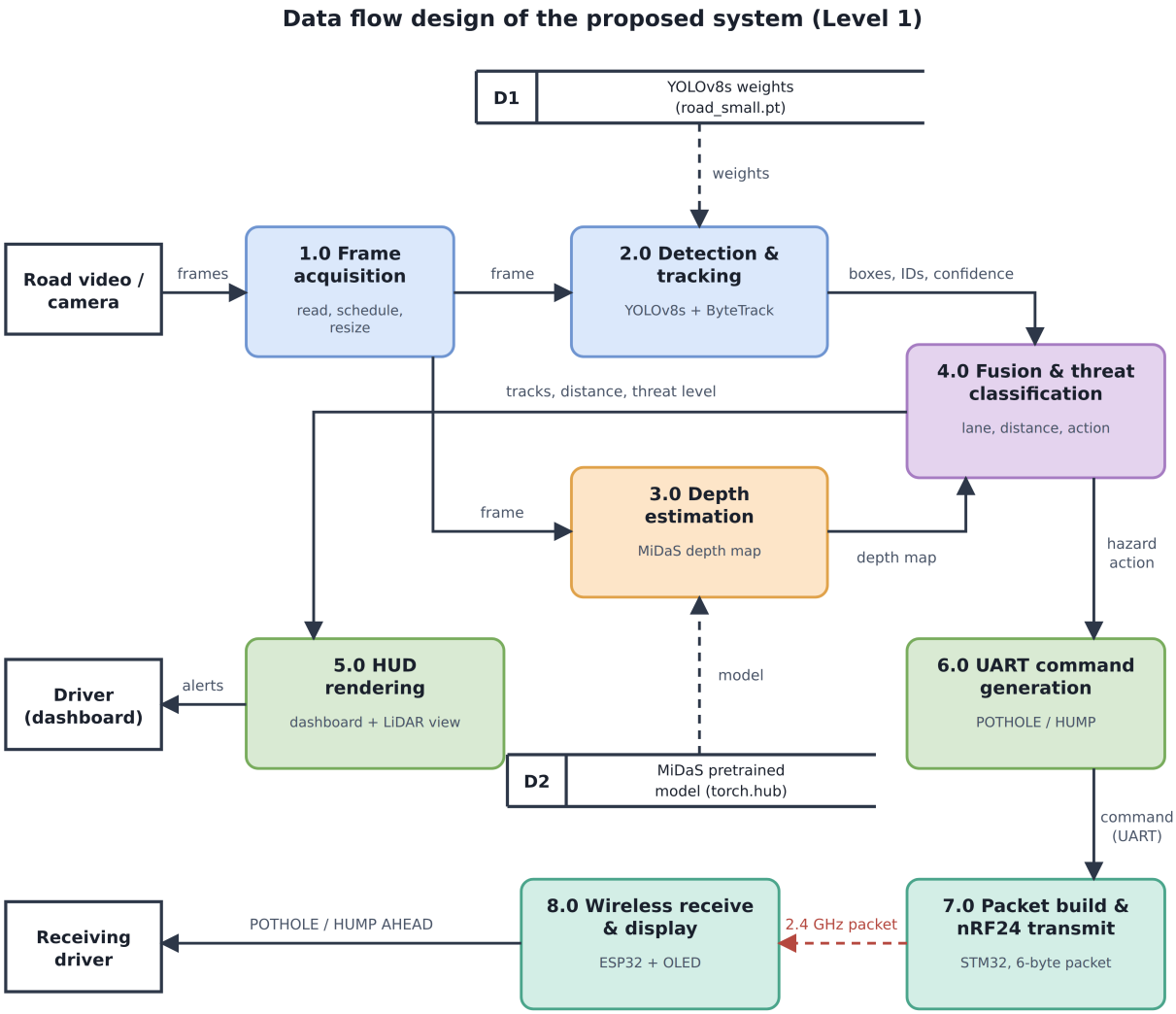

Figure 4.3: Data flow design of the proposed system

4.4 Hardware Design

The hardware consists of three boards and two radio adapters. The STM32 Nucleo-L432KC is connected to the laptop through its ST-LINK USB connector, which provides a virtual COM port (USART2) and power. The nRF24L01 adapter on the STM32 is wired to SPI1 with two extra GPIO lines. On the receiving side the ESP32 uses its SPI bus for the radio and a separate I2C bus for the OLED. Tables 4.1 and 4.2 list the connections, and Figures 4.4 and 4.5 show the wiring.

Because the Nucleo-32 header is labelled with Arduino-style names (D0 to D13) rather than microcontroller pin names, the mapping between firmware pins (PBx) and physical header labels must be followed exactly; an incorrect mapping produces garbage register reads.

Table 4.1: STM32 Nucleo-L432KC to nRF24L01 adapter connections

Signal	STM32 pin	Nucleo header	nRF24L01 pin	Notes
SCK	PB3 (SPI1_SCK)	D13	SCK	Also drives the on-board LED
MISO	PB4 (SPI1_MISO)	D12	MISO	Data from radio to STM32
MOSI	PB5 (SPI1_MOSI)	D11	MOSI	Data from STM32 to radio
CE	PB0 (GPIO)	D3	CE	Starts a transmission
CSN	PB1 (GPIO)	D6	CSN	SPI chip select
VCC	5V	5V	VCC	Adapter with onboard regulator
GND	GND	GND	GND	Common ground

Table 4.2: ESP32 connections to the nRF24L01 adapter and the SSD1306 OLED

Signal	ESP32 pin	Device pin	Notes
SCK	GPIO 18	nRF24 SCK	SPI clock
MISO	GPIO 19	nRF24 MISO	Data from radio
MOSI	GPIO 23	nRF24 MOSI	Data to radio
CE	GPIO 4	nRF24 CE	Listening enable
CSN	GPIO 5	nRF24 CSN	Chip select
VCC	5V (VIN)	nRF24 adapter VCC	With 100 μF and 10 μF capacitors
SDA	GPIO 21	OLED SDA	I2C data, address 0x3C
SCL	GPIO 22	OLED SCL / SCK	I2C clock
VDD	3V3	OLED VDD	OLED supply

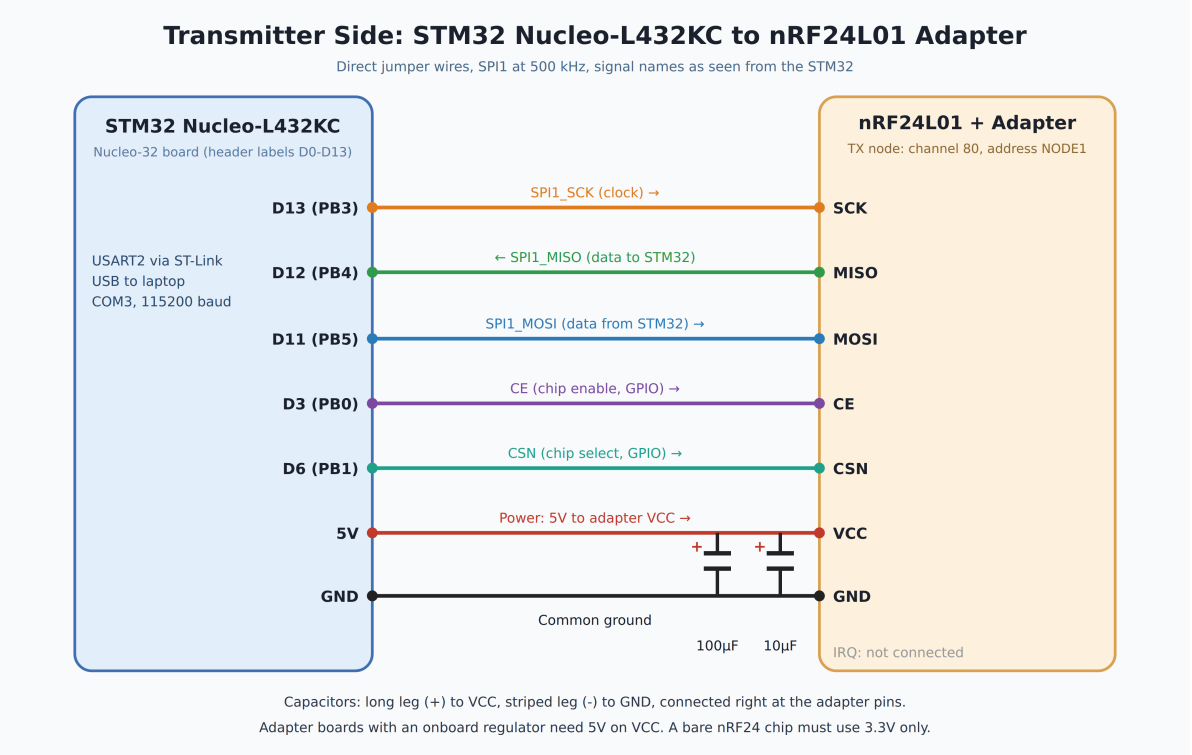

Figure 4.4: Between the STM32 Nucleo-L432KC and the nRF24L01 adapter

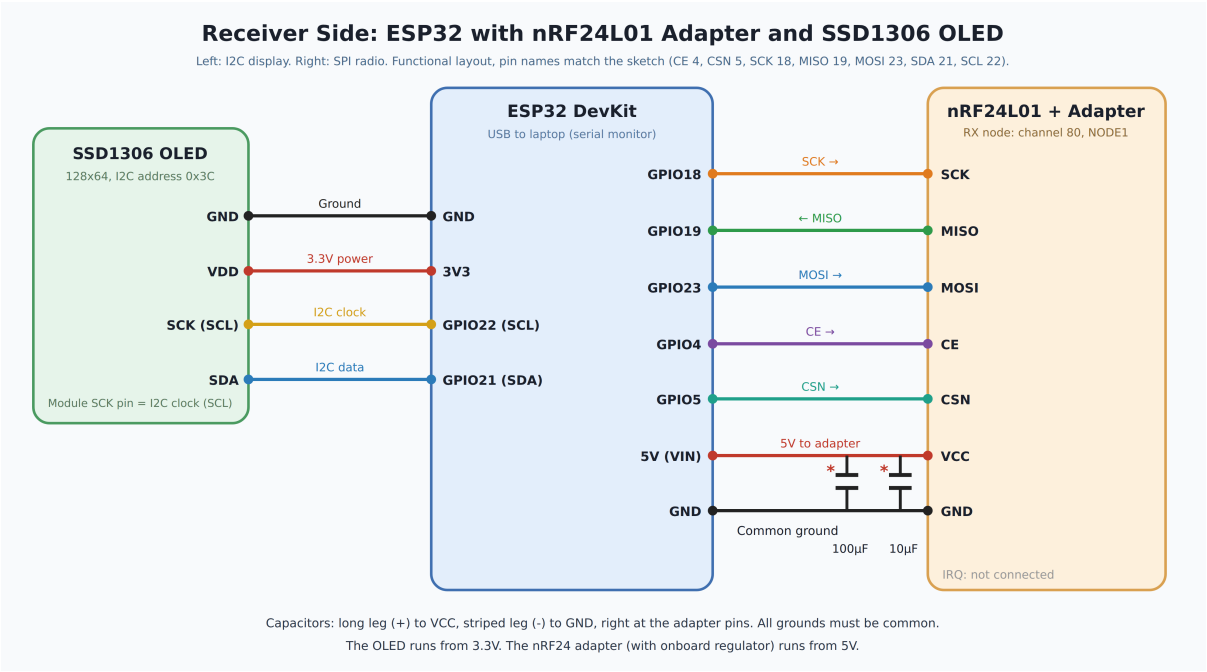

Figure 4.5: Wiring of the ESP32 with the nRF24L01 adapter and the SSD1306 OLED

The nRF24L01 draws short, sharp current spikes when its radio section is active. Both adapters are therefore powered from the 5 V pin, so that the adapter's onboard regulator has enough headroom to produce a stable 3.3 V, and a 100 μF and a 10 μF capacitor are placed across VCC and GND directly at each adapter. The reasons for these choices are explained in Section 6.5.

4.5 Packet and Protocol Design

Two different formats are used. Between the laptop and the STM32 a simple text protocol is used: the command `POTHOLE` or `HUMP` followed by carriage return and line feed at 115200 baud. Between the STM32 and the ESP32 a fixed 6-byte binary packet is used, defined identically in both firmware images as a packed structure so that the compilers of the two different architectures add no padding.

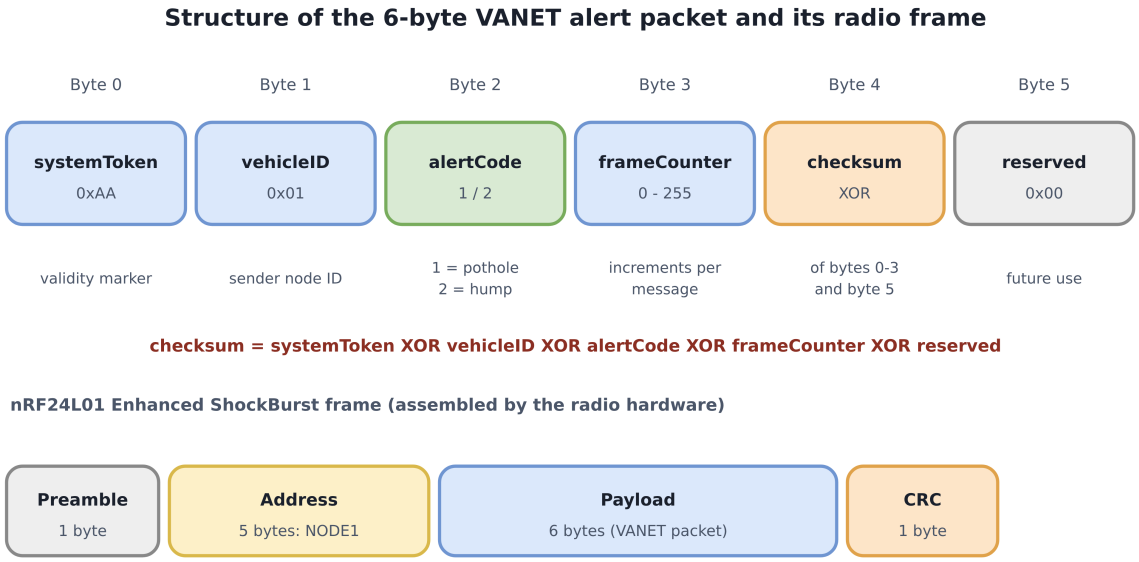

Figure 4.6: Structure of the 6-byte VANET alert packet and its radio frame

The first byte is a fixed system token (0xAA) that lets the receiver reject noise quickly. The alert code identifies the hazard (1 for pothole, 2 for hump). The frame counter increments with each message. The checksum is the XOR of the other five bytes and detects accidental corruption; it is not encryption or authentication. The radio hardware adds the preamble, the 5-byte address NODE1 and a 1-byte CRC around this payload.

Chapter 5

Implementation

5.1 Dataset Preparation and Annotation

A custom dataset of about 1,000 road images was collected, covering three situations: potholes, speed humps, and normal road. Every image was annotated for object detection by drawing bounding boxes around each pothole and hump, and the annotations were exported in YOLO format, in which each image has a text file listing the class ID and the normalised centre x, centre y, width and height of each box.
To make the detector more robust to lighting, viewpoint and surface variation, data augmentation was applied and the dataset was expanded from about 1,000 to about 2,000 images. The augmented dataset was split into training, validation and test subsets 70/20/10 and exported through an API into a Google Colab workspace.

5.2 Model Training on Google Colab

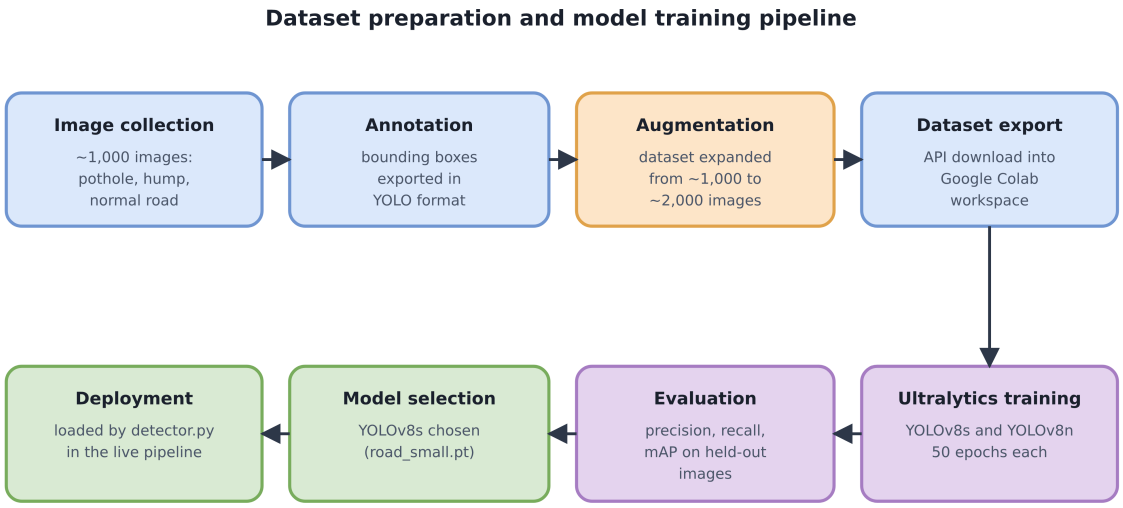

Figure 5.1: Dataset preparation and model training pipeline

Training was carried out with the Ultralytics framework on a Colab GPU. Two detectors, YOLOv8s (small) and YOLOv8n (nano), were fine-tuned from pretrained weights for 50 epochs each on the same dataset. The two models were compared using precision, recall and mAP, and YOLOv8s was selected for deployment and saved as `road_small.pt`. A representative form of the training call is:

```
from ultralytics import YOLO
model = YOLO("yolov8s.pt")
model.train(data="data.yaml", epochs=50)  # same procedure for yolov8n.pt
```

5.3 Detection and Tracking Module

The detector module (`detector.py`) loads the trained weights through Ultralytics and keeps the confidence threshold (0.70) and IoU threshold (0.45) from the configuration file. If the weights file is missing the system reports a warning and continues in an offline bypass mode instead of crashing.
The tracker module (`tracker.py`) calls the Ultralytics tracking interface with ByteTrack and `persist=True`, so that identities are remembered between frames. Only detections that carry a track ID and whose label contains "pothole" or "hump" are passed on as dictionaries holding the ID, label, confidence and bounding box.

```
results = self.detector.model.track(
  source=frame, persist=True, tracker="bytetrack.yaml",
  conf=self.detector.conf_thresh, iou=self.detector.iou_thresh,
  device="cpu", verbose=False)
```

5.4 Virtual LiDAR Depth Module

The depth module (`depth_estimator.py`) loads the MiDaS_small model through `torch.hub`, converts each BGR frame to RGB, applies the MiDaS transform, runs the network on the CPU, resizes the prediction back to the frame size with bicubic interpolation, and normalises it to the range 0 to 255. Higher values mean nearer pixels. The output is used both by the fusion engine and by the separate depth-map debug window.
A geometric fallback based on a flat-ground pinhole model is also implemented. For the bottom edge of a bounding box at image row y_b, a horizon row $y_h = 0.42\,H$, camera height $h = 1.20$ m and focal length $f = 700$ px, the distance is:

$$Z = \frac{h \times f}{y_b - y_h} \tag{5.1}$$

clipped to the range 1.5 m to 15 m. In the current live pipeline the fusion engine uses the blended estimate described in the next section; the pinhole function is kept as a calibrated alternative that can be evaluated in future work.

5.5 Decision-Level Fusion Engine

The fusion engine (`fusion.py`) combines the independent outputs of the detector and of the depth network. It first assigns a lane from the horizontal centre of the box: left if it is in the left 30% of the frame width, right if it is in the right 30%, otherwise centre (the ego lane).
The distance is then estimated in two ways and blended. A geometric estimate uses how low the bottom of the box sits in the frame, and a depth estimate converts the MiDaS value sampled

just above the bottom edge of the box:

$$d_{geo} = 25 \times \left(1 - \frac{y_2}{H}\right) \tag{5.2}$$

$$d_{depth} = \frac{150}{\text{depth value}} \tag{5.3}$$

$$d = \text{clip}\left(0.6\, d_{geo} + 0.4\, d_{depth},\ 1,\ 30\right)\ \text{metres} \tag{5.4}$$

A tiny constant is added to the depth value in the code only to avoid division by zero. If the box bottom is in the lowest 15% of the frame, the hazard is treated as immediately ahead and the distance is set to 1.0 m. The constants 25 and 150 are tuning values chosen for this camera setup; they are not derived from camera calibration, which is why the distance is described as an approximate fused estimate and not a measurement.

The threat level depends on the lane and distance. A centre-lane hazard at 4.5 m or closer (or whose box bottom is in the lowest 20% of the frame) is `EMERGENCY_BRAKE`; one within 9.5 m is `SLOW_DOWN`; anything else, and every hazard in a neighbouring lane, is `MONITOR`. A hysteresis latch then compares the result with the previous action for the same track: if the hazard was already at `EMERGENCY_BRAKE` or `SLOW_DOWN` and is within 6 m, the higher state is kept, so a noisy depth value cannot make the alarm flicker off. The fusion engine also computes an agreement flag that is YES only when the detector confidence is at least 0.70 and the proximity verdict is not safe.

A short blind-spot memory in the main loop keeps an `EMERGENCY_BRAKE` or `SLOW_DOWN` alert alive for 15 frames after a very close hazard (3.5 m or less) leaves the camera view.

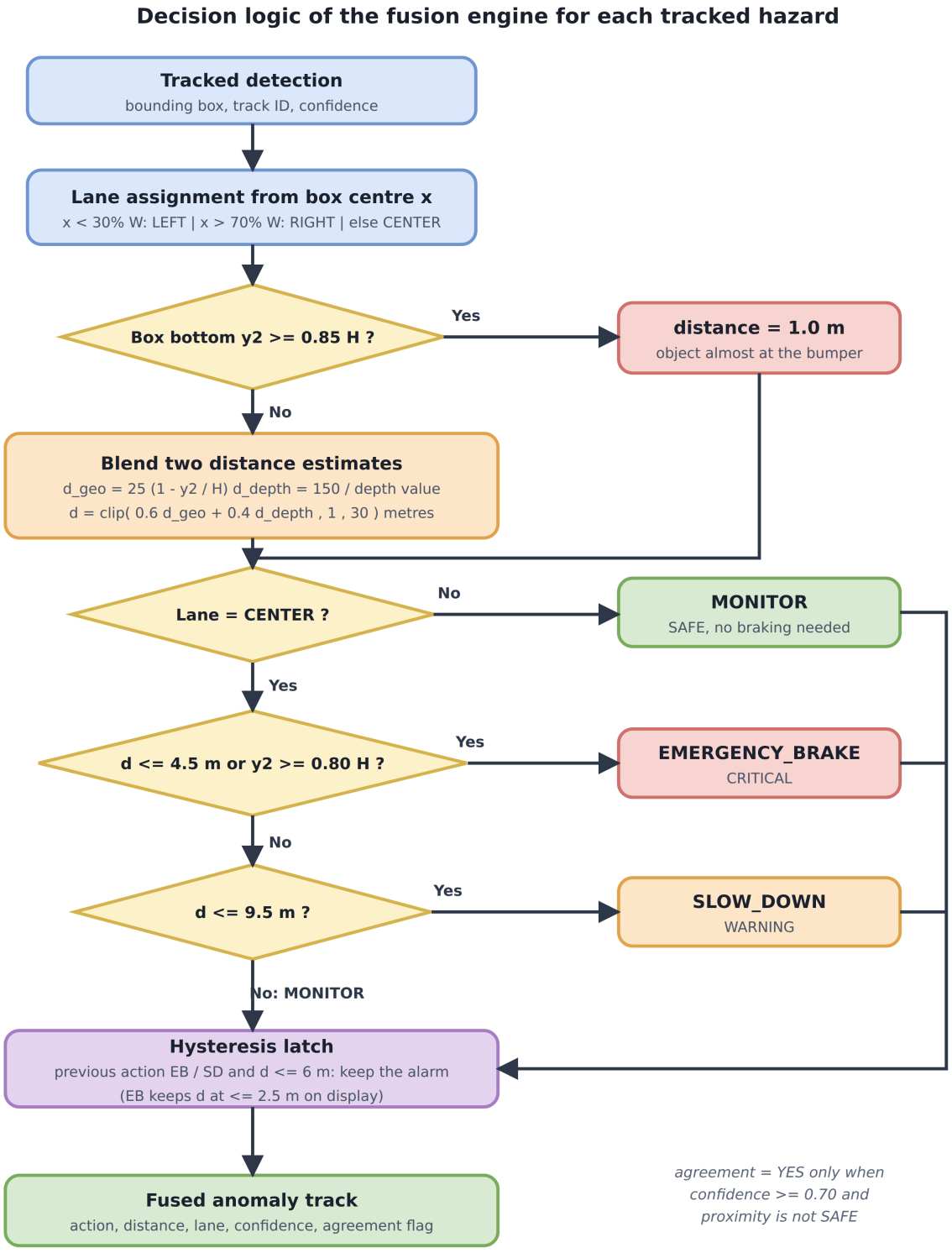

Figure 5.2: Logic of the fusion engine for each tracked hazard

Table 5.1: Main fusion and pipeline parameters

Parameter	Value	Meaning
Confidence threshold	0.70	Minimum detector confidence
IoU threshold	0.45	Overlap used to merge duplicate boxes
Inference stride	2	Heavy models run on every second frame
Lane boundaries	30% and 70% of width	Left / centre / right assignment
Blend weights	0.6 geometric, 0.4 depth	Distance estimate (Eq. 5.4)
Emergency brake distance	4.5 m (centre lane)	Critical threshold
Slow-down distance	9.5 m (centre lane)	Warning threshold
Hysteresis distance	6.0 m	Keeps a previous high-threat state
Blind-spot hold	15 frames	Keeps alert after hazard leaves view
UART resend interval	1.5 s	Minimum time between commands

5.6 Dashboard and Visualisation

The dashboard module (`hud.py`) renders a 1280 by 720 canvas split into the annotated road feed and a telemetry panel, as in Figure 5.4. Each tracked hazard is drawn with corner brackets and a label showing its ID, class, distance and state, coloured red for `EMERGENCY_BRAKE`, amber for `SLOW_DOWN` and green for `MONITOR`. A flashing bar appears across the top for `EMERGENCY_BRAKE`. The panel contains a status card, a session hazard counter that shows the number of potholes and humps seen (each unique track counted once), the fused range of the nearest hazard, a recommended driver action, and a rolling log of recent detections. The recommended action reads CLEAR ROAD when nothing needs action, BRAKE for an emergency in the ego lane, SLOW DOWN for a warning, and TURN RIGHT or TURN LEFT for an emergency hazard in the left or right lane respectively. A second window shows the MiDaS depth map coloured only inside the tracked boxes.

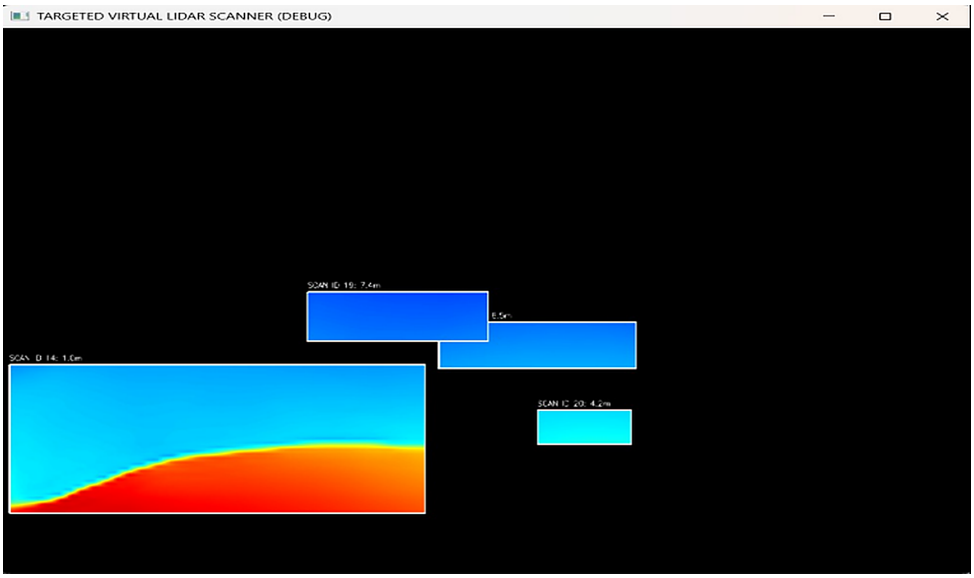

Figure 5.3: Layout of the virtual LiDAR window

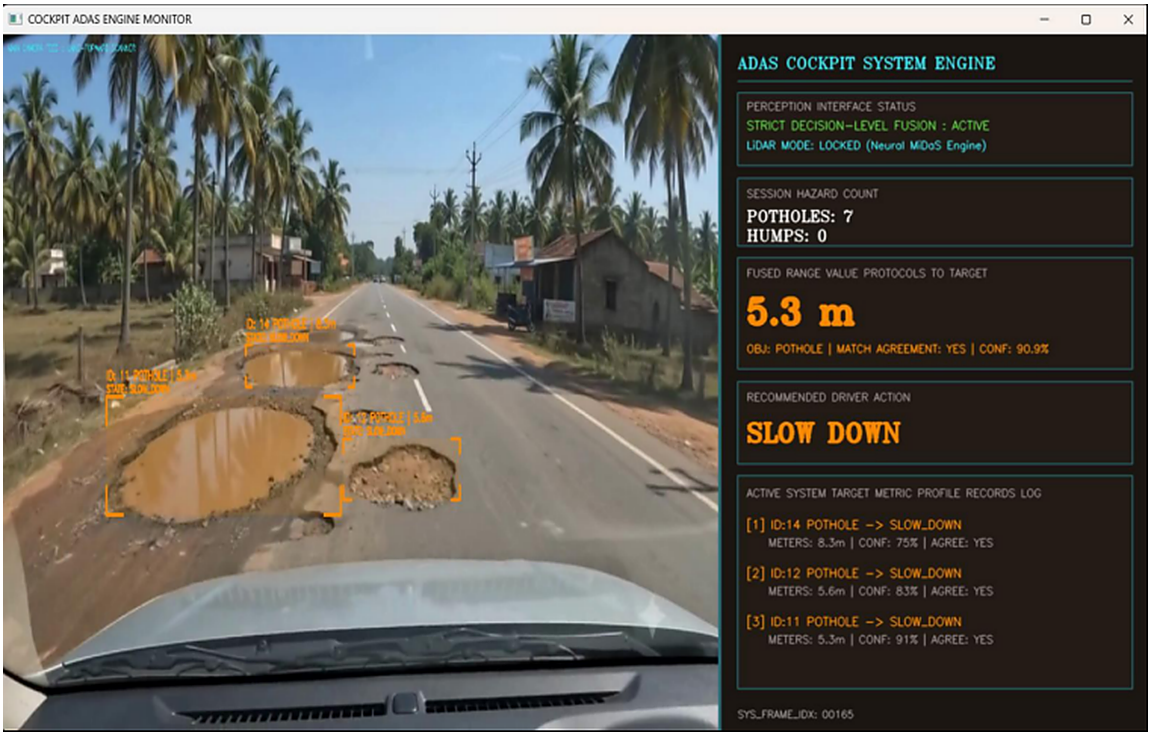

Figure 5.4: Layout of the cockpit dashboard

5.7 Software-to-Hardware Bridge

The main loop sends a command to the STM32 whenever the nearest hazard is at `EMERGENCY_BRAKE` or `SLOW_DOWN`. The label of the hazard decides the text: `HUMP` if the label contains "hump", otherwise `POTHOLE`. The command is terminated with carriage return and line feed, because the STM32 firmware only acts on a complete line. To avoid flooding the radio, commands are sent at most once every 1.5 seconds.

At start-up the program opens the ST-LINK virtual COM port with pyserial at 115200 baud and sends a terminated test line, so that no stray character remains in the STM32 receive buffer and corrupts the first real command. If the port cannot be opened, the program continues in simulation mode.

```
if ser and ser.is_open and hazard_command:
  if (current_time − last_uart_send_time) >= CFG.uart_resend_interval_s:
    ser.write(f"{hazard_command}\r\n".encode("utf−8"))
    ser.flush()
    last_uart_send_time = current_time
```

5.7.1 STM32 Firmware

The STM32 firmware was developed in STM32CubeIDE with the HAL library. The system clock is derived from the 4 MHz MSI oscillator multiplied by a PLL (M = 1, N = 16, R = 2) to 32 MHz, with one flash wait state and all bus dividers set to one. USART2 receives characters through an interrupt: `HAL_UART_RxCpltCallback()` appends each byte to a buffer, and on carriage return or line feed compares the buffer with "POTHOLE" and "HUMP", setting a command type and a ready flag. The ISR does no radio work; the main loop polls the flag, builds the packet, and sends it, which keeps the interrupt short.

Table 5.2: STM32 peripheral configuration

Peripheral	Setting	Value
System clock	MSI 4 MHz, PLL M=1 N=16 R=2	32 MHz, Voltage scale 1, flash latency 1
USART2	Pins PA2 (TX), PA15 (RX)	115200 baud, 8N1, interrupt receive, priority 0
SPI1	Pins PB3 (SCK), PB4 (MISO), PB5 (MOSI)	Master, mode 0, 8-bit, MSB first, software NSS, prescaler 64 (500 kHz)
GPIO	PB0 (CE), PB1 (CSN)	Push-pull outputs, low speed

5.7.2 nRF24L01 Driver and Radio Configuration

The driver (`nrf24.c`) implements register read and write, buffer write, FIFO flush, status read, initialisation and send functions over SPI1. Every transaction pulls CSN low, shifts a command byte and data, and releases CSN. The initialisation writes the registers in Table 5.3. Auto-Acknowledge is deliberately disabled: with several possible receivers, acknowledgements from each would collide, and the sender does not need to know who received the alert.

Table 5.3: nRF24L01 register configuration on the transmitter

Register	Value	Meaning
CONFIG	0x0A	CRC enabled (1 byte), powered up, transmit mode
EN_AA	0x00	Auto-Acknowledge disabled (broadcast)
EN_RXADDR	0x01	Pipe 0 enabled
SETUP_AW	0x03	5-byte addresses
SETUP_RETR	0x00	No automatic retransmission
RF_CH	0x50 (80)	Channel 80, 2.480 GHz
RF_SETUP	0x06	1 Mbps, 0 dBm
TX_ADDR / RX_ADDR_P0	NODE1	Shared 5-byte address
RX_PW_P0	6	Payload size = packet size

Hardware testing showed that a single register write could be corrupted when the supply or the SPI signal was marginal, most seriously the PWR_UP bit of CONFIG, which leaves the radio unable to transmit even though the firmware proceeds normally. The driver therefore waits 100 ms after power-up, verifies that PWR_UP is set (with up to five retries), and writes every other configuration register through a write-and-read-back routine with a 2 ms settle delay.
To send a packet, the 6 bytes are loaded into the TX FIFO with the `W_TX_PAYLOAD` command, CE is pulsed high for 1 ms to start the transmission, and the STATUS register is polled for the TX_DS flag with a 100 ms timeout. Because no acknowledgement is requested, each alert is transmitted three times, 5 ms apart, to reduce the chance that a single lost transmission is missed.

5.7.3 ESP32 Receiver and OLED Display

The ESP32 sketch uses the RF24 library with the same channel (80), data rate (1 Mbps), 8-bit CRC, 5-byte address (NODE1) and payload size (6) as the transmitter, and Auto-Acknowledge disabled. The SPI clock is set to 1 MHz for better signal integrity. In the main loop the sketch checks `radio.available()`, reads the payload, and rejects it unless the token is 0xAA and the recomputed XOR checksum matches. For a valid packet it draws a bordered screen with the vehicle ID and frame counter and the text POTHOLE AHEAD or HUMP AHEAD in two lines (the text is split because 13 characters at text size 2 are wider than the 128-pixel display). After three seconds without a new packet it returns to the waiting screen.

5.8 Power and Signal-Integrity Measures

Several measures were needed to make the radio links reliable, and they are summarised here because they are a significant part of the hardware design. Each adapter is powered from 5 V so that its onboard regulator has headroom; a 100 μF and a 10 μF capacitor are placed across VCC and GND at each adapter to supply the current spikes of the radio; the SPI lines use short direct jumper wires instead of a breadboard; the STM32 SPI clock was reduced from 2 MHz to 500 kHz and the ESP32 SPI clock set to 1 MHz; and the configuration registers are written with read-back verification. The problems that motivated each measure are listed in Section 6.5.

Chapter 6

System Testing

6.1 Test Environment and Test Data

The perception pipeline was tested on a Windows laptop running Python with OpenCV, PyTorch and Ultralytics; both YOLO and MiDaS were executed on the CPU 12500H i5 12th gen and 16GB RAM. The test video was a recorded road video containing potholes, humps and clear road. The embedded tests used the STM32 Nucleo-L432KC and the ESP32 DevKit described in Chapter 4, with PuTTY as the serial terminal at 115200 baud and the Arduino IDE serial monitor for the ESP32.

The detector was evaluated on 250 held-out images that were not used for train-ing. The distance estimate was checked against distances measured with a tape on a marked test stretch, and the wireless chain was checked with manual commands and with the full pipeline.

6.2 Detection Evaluation Metrics

Detection quality was measured using precision, recall, F1-score and mean average precision (mAP). A predicted box is counted as correct when its Intersection over Union (IoU) with a true box exceeds a threshold, where P is the predicted box and G the ground-truth box:

$$\text{IoU} = \frac{\text{area}(P \cap G)}{\text{area}(P \cup G)} \tag{6.1}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{6.2}$$

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{6.3}$$

where TP, FP and FN are the numbers of true positives, false positives and false negatives. The average precision (AP) of a class is the area under its precision–recall curve, and mAP is the mean of the AP values over all classes. mAP@0.5 uses an IoU threshold of 0.5, while mAP@0.5:0.95 averages over thresholds from 0.5 to 0.95.

6.3 Distance Validation Method

To judge how far the fused distance can be trusted, a hazard (or a hazard marker) is placed at known distances in front of the camera, for example 2, 4, 6, 8 and 10 m, and the distance shown

on the dashboard is recorded. The mean absolute error over n measurements is:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|d_{fused,i} - d_{true,i}\right| \tag{6.4}$$

Because the MiDaS output is relative and the conversion constants are tuned values, this test is important: it shows the range over which the fused distance is accurate enough to support the `EMERGENCY_BRAKE` and `SLOW_DOWN` thresholds.

6.4 Embedded and Wireless Link Testing

The embedded chain was tested in stages so that a failure could be located precisely: the UART command path, the STM32-to-radio SPI link (by printing the nRF24L01 registers at boot), the wireless hop, and the ESP32 validation and display. Manual commands typed into the serial terminal were used before the Python pipeline was connected, so that the hardware could be tested independently. Table 6.1 lists the test cases.

Table 6.1: Embedded and system test cases

ID	Test	Expected result	Status
TC1	Type POTHOLE in the serial terminal	STM32 prints "Sending POTHOLE! ID=1 Frame=n Check=0x.."	Pass
TC2	Boot register dump of the transmitter radio	CONFIG 0x0A, EN_AA 0x00, EN_RXADDR 0x01, SETUP_AW 0x03, SETUP_RETR 0x00, RF_CH 0x50, RF_SETUP 0x06, STATUS 0x0E	Pass
TC3	ESP32 boot	Radio initialised, waiting screen on the OLED	Pass
TC4	Wireless reception	ESP32 prints PACKET RECEIVED with token 0xAA, ID 1, alert 1 or 2	Pass
TC5	OLED alert	POTHOLE AHEAD / HUMP AHEAD for 3 s, then waiting screen	Pass
TC6	Invalid payload rejection	Zero or garbage payloads print INVALID TOKEN and raise no alert	Pass
TC7	Perception pipeline on the test video	Boxes, IDs, distances, counters and driver action displayed	Pass
TC8	Command throttling	No more than one UART command every 1.5 s	Pass
TC9	End-to-end latency	Time from command to OLED update measured	30-40ms

6.5 Hardware Integration Issues and Resolutions

Integrating the radio hardware was the most demanding part of testing. Table 6.2 records the problems that were observed, their causes, and the resolutions adopted. The issues were found

by reading back the nRF24L01 registers at boot, because the firmware otherwise printed a normal “Sending” message even when the radio was not working.

Table 6.2: Hardware integration issues and resolutions

Observed problem	Cause	Resolution
STM32 never forwarded alerts while the Python program was running	The program sent single characters without a line end, but the firmware acts only on a complete line POTHOLE or HUMP	Program now sends POTHOLE or HUMP followed by CR LF
First real command rejected after start-up	A stray start-up test byte stayed in the receive buffer and was prepended to the command	Start-up test line is terminated with CR LF so the buffer is cleared
Random symbols in the serial terminal	Wrong baud rate or wrong COM port	115200 baud on the ST-LINK COM port; port fixed in the program
CONFIG read 0x08 instead of 0x0A while other registers were correct	PWR_UP bit lost by a corrupted write on a marginal supply, so the radio could not transmit	100 ms power-up delay and verify-and-retry on PWR_UP
All registers read 0x00 and the adapter LED was off	No power reached the radio adapter	Power and ground connections restored; direct wiring used
Scrambled registers that changed between two reads	Adapter regulator lacked headroom at 3.3 V input; supply spikes; breadboard contact	Adapter powered from 5 V; 100 μF and 10 μF capacitors; direct jumper wires
RF_CH read 160 instead of 80 (bits shifted by one)	SPI timing margin at 2 MHz on jumper wires	SPI clock reduced to 500 kHz; every write verified by read-back
Unexpected register values although the wiring looked correct	Nucleo-32 header labels (D13, D12, D11, D3, D6) differ from microcontroller pin names	Pin-by-pin mapping to PB3, PB4, PB5, PB0, PB1 verified
Some alerts not received	With Auto-Acknowledge off there is no hardware retransmission	Each alert is transmitted three times, 5 ms apart
ESP32 showed NRF ERROR after an edit	The three-argument RF24 constructor was not compatible with the manual `SPI.begin()` call in this library version	Two-argument constructor restored; SPI speed set with `SPI.setFrequency(1 MHz)`
Receiver printed packets whose bytes were all identical or zero	Unstable SPI read on the receiver; the token check rejects such payloads	Token and checksum validation; RX FIFO flush; slower SPI. [final status]

6.6 Dashboard and Pipeline Testing

Road-video tests confirmed persistent hazard IDs, one count per hazard, distance-based CLEAR ROAD/SLOW DOWN/BRAKE actions, lane-based emergency turns, alerts retained by hysteresis and blind-spot memory, and depth data limited to tracked boxes.

6.7 Testing Summary

The tests confirmed that the UART command path, the radio register configuration, and the receiver boot and display work, and they exposed the power-supply and signal-integrity weaknesses of a prototype built on jumper wires. After the measures listed in Table 6.2 the transmitter radio reads back the expected configuration. The end-to-end reception and latency results are reported in Chapter 7.

Chapter 7

Results and Discussion

7.1 Detection Results

The YOLOv8s and YOLOv8n detectors were trained for 50 epochs on the same augmented dataset and evaluated on the held-out images. Table 7.1 summarises their results. YOLOv8s was selected for deployment as the better accuracy trade-off for a laptop-CPU pipeline.

Table 7.1: Detection results on the held-out test set

Model	**Precision**	**Recall**	**mAP@0.5**	**mAP@0.5:0.95**	**Inference time (CPU)**
YOLOv8s (deployed)	0.90	0.37	0.40	0.23	40–70 ms
YOLOv8n	0.85	0.34	0.40	0.20	25–45 ms

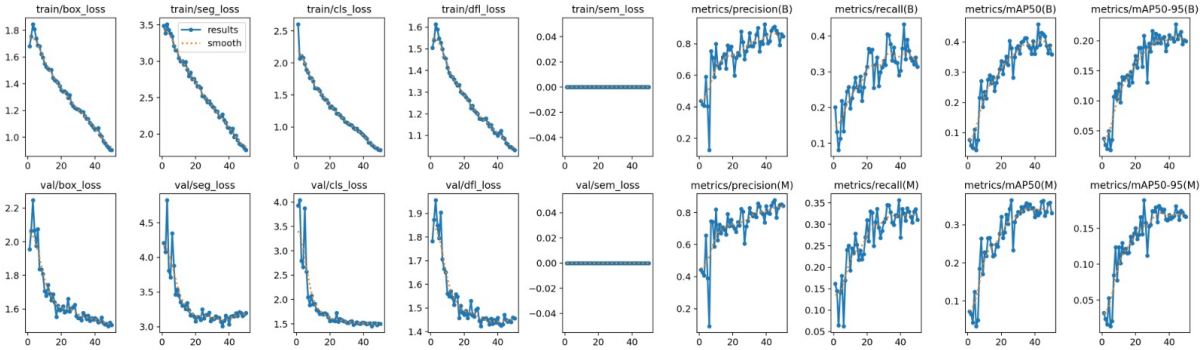

Figure 7.1: Curves of the deployed YOLOv8s model

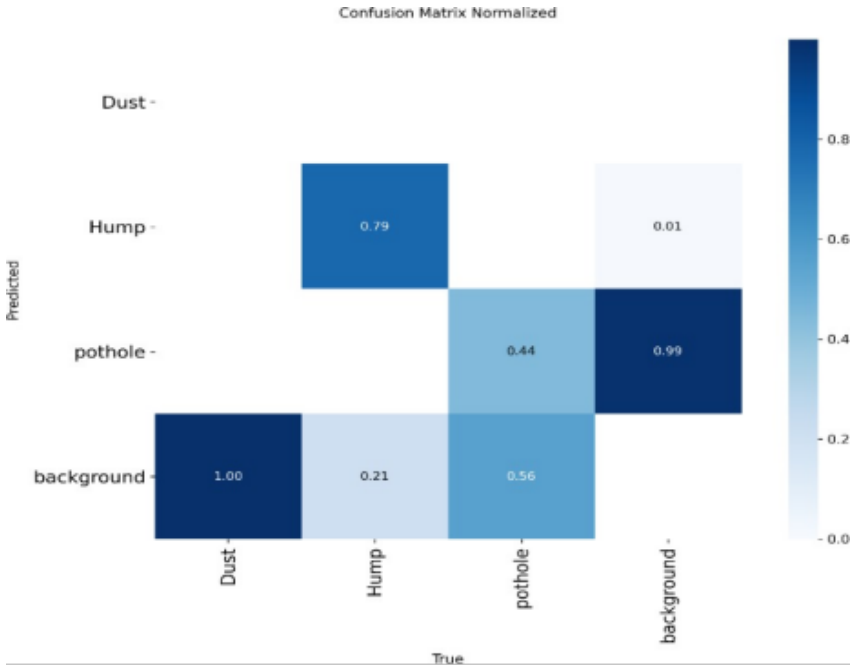

Figure 7.2: Confusion matrix of the deployed YOLOv8s model

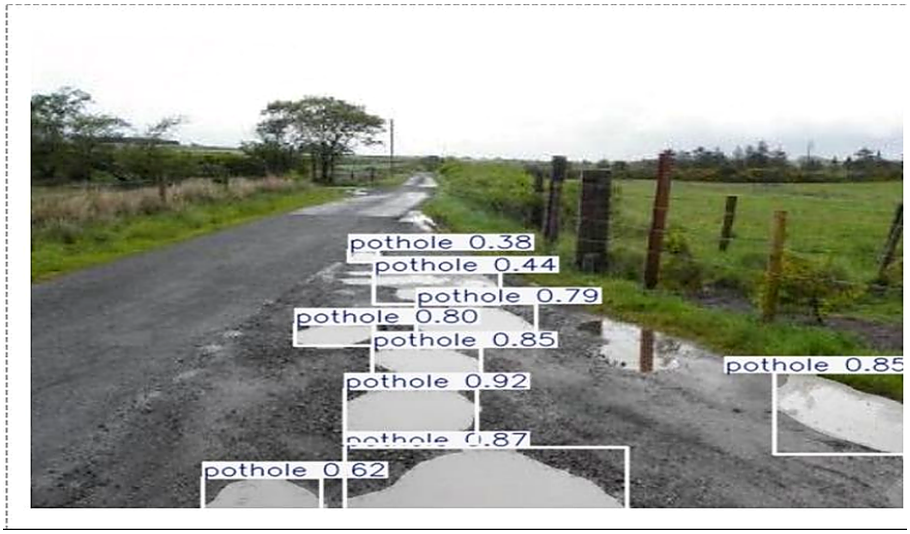

Figure 7.3: Sample detections of potholes on road images

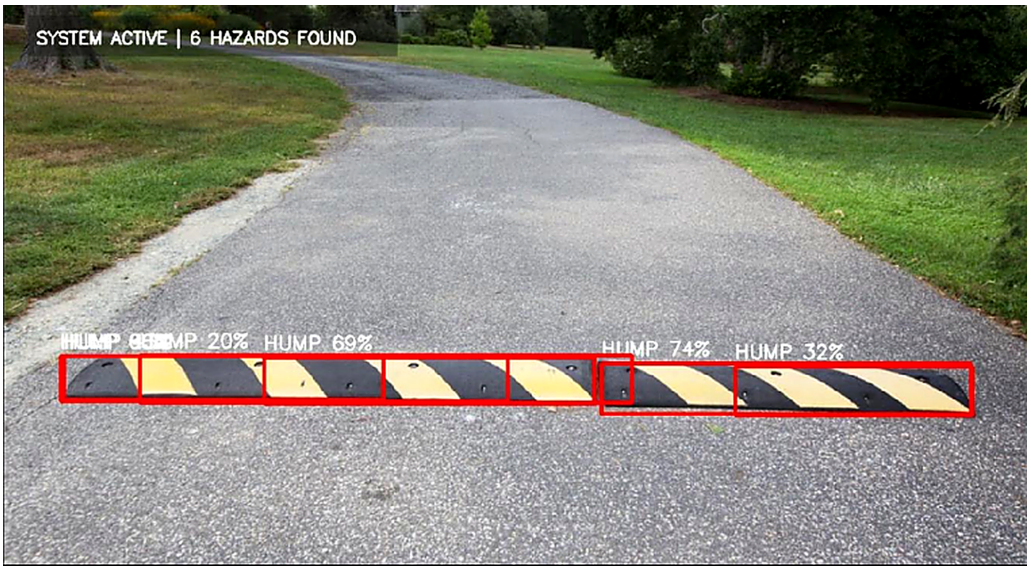

Figure 7.4: Sample detections of humps on road images

7.2 Wireless Chain Results

The transmitter radio, after the power and signal-integrity measures, reads back the expected register values (Table 6.1, TC2). Table 7.2 summarises the end-to-end behaviour of the alert chain.

Table 7.2: End-to-end results of the alert chain

Measure	**Result**
Radio configuration at boot	CONFIG 0x0A, EN_AA 0x00, RF_CH 0x50 (channel 80), RF_SETUP 0x06
Alerts sent by the STM32 in the trial	20
Alerts received and verified by the ESP32	19
Reception rate	95%
Time from UART command to OLED update	35-45ms
Indoor / outdoor range tested	20-25m

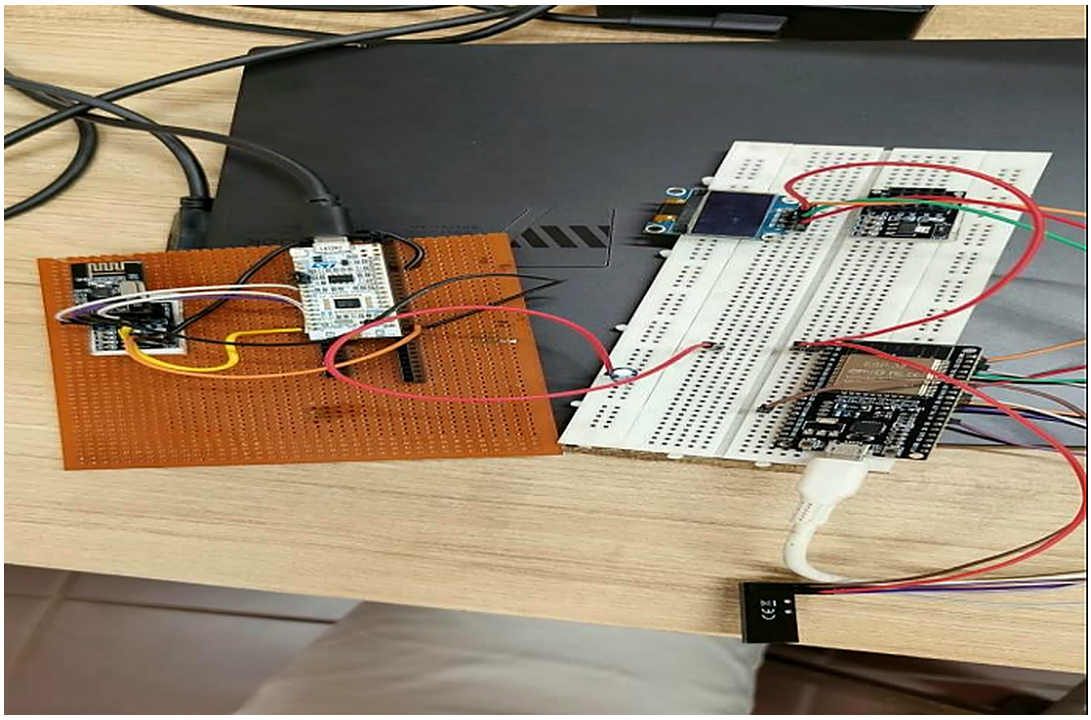

Figure 7.5: Complete hardware setup of the transmitter and receiver nodes

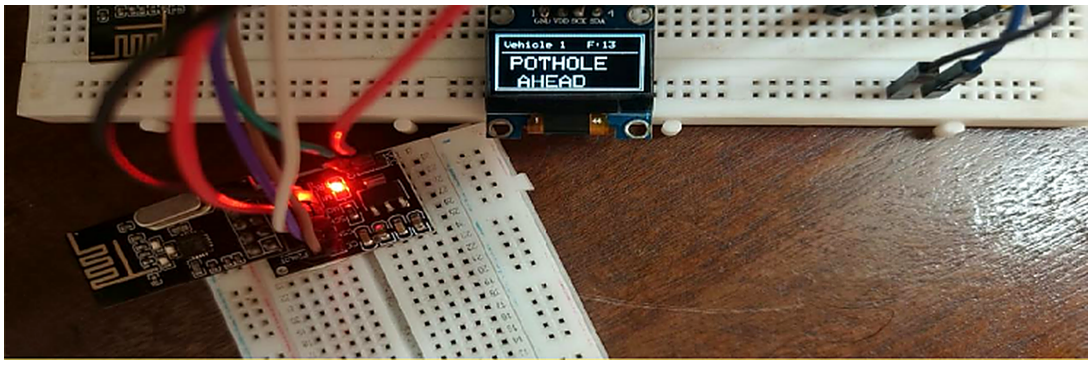

Figure 7.6: Receiver OLED displaying the hazard alerts, Example 1

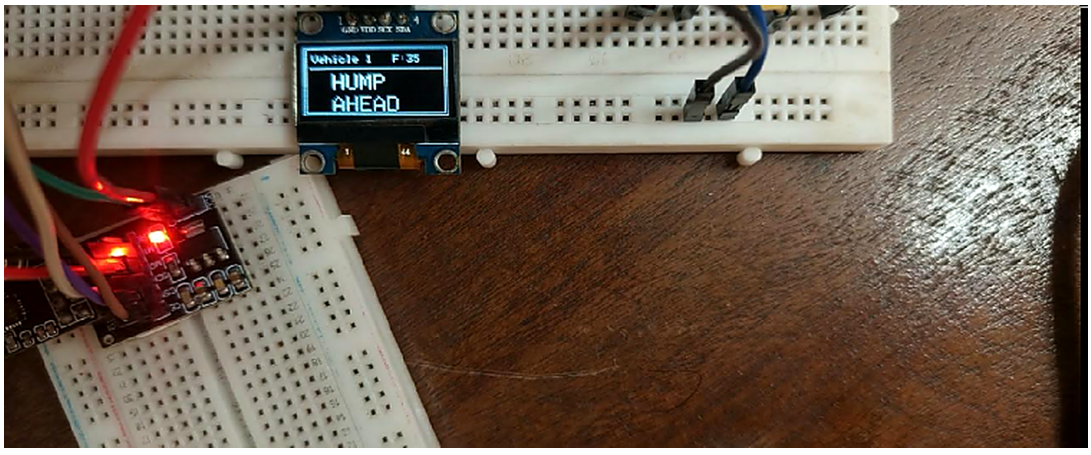

Figure 7.7: Receiver OLED displaying the hazard alerts, Example 2

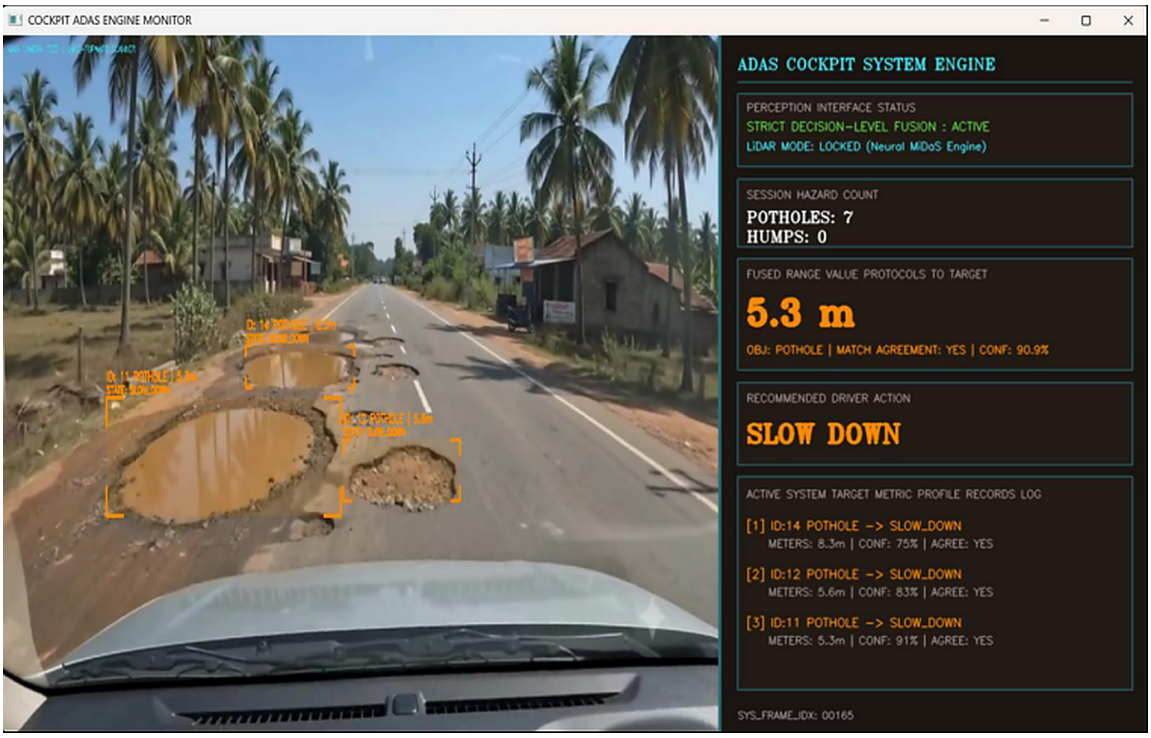

Figure 7.8: Cockpit dashboard and virtual LiDAR window during a detection

7.3 Discussion

Detection. The custom-trained YOLOv8s detector forms the foundation of the system, and its quality depends on how well the 2,000-image dataset covers real road conditions. Rain, night driving, shadows and unusual road textures are likely to be under-represented in any small dataset, and the metrics in Table 7.1 should be read with that in mind.

Depth and fusion. MiDaS provides a relative depth map, not a metric one. The fused distance therefore relies on tuned constants (25 and 150) and on the 60/40 blend, which were chosen for this camera setup and have not been derived from camera calibration. The agreement flag is informational: it is shown on the dashboard but does not gate the braking decision. The fusion is a rule-based decision-level fusion with temporal hysteresis, not a probabilistic or learned fusion. The pinhole projection formula implemented in the depth module is a natural candidate for a calibrated replacement.

Hardware. The prototype showed how sensitive the nRF24L01 is to its supply and wiring. A radio can appear healthy to the firmware while a single corrupted register write leaves it unable to transmit, so register read-back at boot proved to be an essential diagnostic. Decoupling capacitors, a 5 V feed for the adapter regulator, direct wiring and a slower SPI clock all contributed to stability.

Relation to real V2V. The wireless layer reproduces the broadcast hazard-warning pattern but is not a V2V standard implementation. Table 7.3 lists the differences.

Table 7.3: Comparison between this prototype and deployed V2V systems

Aspect	**Deployed V2V (DSRC / C-V2X)**	**This project**
Spectrum	Dedicated 5.9 GHz ITS band or cellular V2X	Shared 2.4 GHz ISM band (nRF24L01)
Message format	Standardised (SAE J2735 Basic Safety Message)	Custom 6-byte packet
Position data	GPS position, heading, speed in every message	None; receiver knows only that a hazard exists
Topology	Every vehicle both sends and receives	One transmitting node, one or more receiving nodes
Security	Signed messages and certificates (IEEE 1609.2)	XOR checksum for corruption only
Range	Hundreds of metres	Tens of metres with the bare module
Certification	Automotive-grade, regulated	Hobby-grade, uncertified

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project developed an end-to-end prototype that detects potholes and speed humps from road video, estimates their distance, recommends a driver action, and broadcasts a wireless alert to a separate display node. A custom dataset of about 1,000 images was annotated and augmented to about 2,000 images, and YOLOv8s and YOLOv8n detectors were trained for 50 epochs, with YOLOv8s deployed. ByteTrack provides persistent identities, so each hazard is shown and counted once.

Because a single camera does not measure range, MiDaS monocular depth was used as a virtual LiDAR and fused at decision level with the detector output and lane position. The fusion engine blends a position-based heuristic with the depth value, classifies each hazard as `EMERGENCY_BRAKE`, `SLOW_DOWN` or `MONITOR`, and uses hysteresis and a short blind-spot memory to keep alerts stable. The cockpit dashboard shows the annotated feed, the fused distance, the session pothole and hump counts, the recommended action, and a separate depth window.

On the embedded side, a Python-to-STM32 UART protocol, a 6-byte packet with token and checksum, an nRF24L01 broadcast with Auto-Acknowledge disabled and triple transmission, and an ESP32 receiver with an OLED alert display were implemented. Hardware testing identified and resolved a series of power-supply, wiring and signal-integrity problems, and showed the value of reading back radio registers at boot.

The system is an academic prototype. It demonstrates the broadcast pattern of V2V hazard warning but does not implement the V2V standards, GPS positioning or message authentication, and its distance estimate is approximate. It must not be used as a substitute for driver judgement or a certified safety system.

8.2 Future Work

The first priority is to extend and diversify the dataset with night, rain, shadow and different road types, and to evaluate the detector on independent road videos. Larger models, or export of the detector to a GPU or an embedded accelerator, would raise the frame rate beyond what a laptop CPU allows.

The distance estimate can be improved by calibrating the camera and replacing the tuned constants with the pinhole ground-plane model already implemented, by sampling depth over the whole bounding box instead of one pixel, by fine-tuning a depth model on road scenes, or by adding a physical range sensor such as a radar or LiDAR. The agreement flag could be made to gate the braking decision, and the system could fall back to camera-only operation if the depth model is unavailable.

The wireless layer can be brought closer to real V2V by adding a GPS module so that every packet carries the hazard's position and time, by making every node both a transmitter and a receiver, by adding a message authentication code to prevent forged alerts, and by moving to a longer-range radio such as LoRa or to certified DSRC or C-V2X hardware. Field trials on a moving vehicle, with measured range and latency, would be needed before the system could be considered for any practical use.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," software, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] Y. Zhang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *Computer Vision – ECCV 2022*, 2022.

[4] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, 2022.

[5] A. Dhiman and R. Klette, "Pothole Detection Using Computer Vision and Learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 8, pp. 3536–3550, 2020.

[6] S. Sattar, S. Li, and M. Chapman, "Road Surface Monitoring Using Smartphone Sensors: A Review," *Sensors*, vol. 18, no. 11, art. 3845, 2018.

[7] A. Geiger, P. Lenz, and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3354–3361, 2012.

[8] J. B. Kenney, "Dedicated Short-Range Communications (DSRC) Standards in the United States," *Proceedings of the IEEE*, vol. 99, no. 7, pp. 1162–1182, 2011.

[9] SAE International, "Dedicated Short Range Communications (DSRC) Message Set Dictionary," SAE Standard J2735.

[10] IEEE, "IEEE Standard for Wireless Access in Vehicular Environments – Security Services for Applications and Management Messages," IEEE Std 1609.2.

[11] Nordic Semiconductor, "nRF24L01+ Single Chip 2.4 GHz Transceiver Product Specification."

[12] STMicroelectronics, "STM32 Nucleo-32 boards (MB1180), User Manual UM1956."

[13] Espressif Systems, "ESP32 Series Datasheet."

[14] Solomon Systech, "SSD1306 128 × 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller, Datasheet."

[15] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.

[16] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

SUSTAINABLE DEVELOPMENT GOALS

Introduction

The United Nations introduced 17 Sustainable Development Goals (SDGs) as part of the 2030 Agenda for Sustainable Development. These goals aim to address global challenges, including public health, infrastructure development, sustainable cities, environmental protection and technological innovation.

Sustainable Development Goals Addressed by the Project

The proposed Intelligent Road Anomaly Detection System addresses the following Sustainable Development Goals:

SDG 3 – Good Health and Well-being

The project contributes to road safety by detecting road anomalies such as potholes and speed humps using computer vision techniques. The system estimates the approximate distance of detected hazards and generates suitable warning alerts. These alerts are transmitted wirelessly to a compatible receiver, helping approaching drivers become aware of potential road hazards. This approach aims to reduce the risk of accidents and improve road-user safety.

SDG 9 – Industry, Innovation and Infrastructure

The project promotes technological innovation by integrating artificial intelligence, embedded systems and wireless communication. It uses YOLO-based object detection for road anomaly identification, STM32 for alert processing, nRF24L01 modules for wireless communication and ESP32 for receiving and displaying hazard alerts. This integration demonstrates the application of modern electronic technologies in developing intelligent transportation solutions.

SDG 11 – Sustainable Cities and Communities

The project supports the development of safer transportation systems by identifying road hazards and broadcasting warnings to vehicles equipped with compatible receivers. It demonstrates the potential of intelligent road-monitoring and vehicle-to-vehicle-style communication systems to improve road awareness and contribute to safer urban mobility.